Palouse Alliance Community Resource Platform

Final Report

Palouse Alliance Team

Mentor
Parteek Kumar

Palouse Alliance for Healthy Individuals, Families, and Communities

Omar Adil, Vijay Deva, Kaeden Peterson, Aryan Puthran

CptS 423 Software Design Project II

Fall 2026

Table of Contents

I. Introduction **4**
I.1. Project Introduction 4
I.2. Background and Related Work 4
I.3. Project Overview 5
II. Team Members & Bios **6**
II.1. Omar Adil 6
II.2. Vijay Deva 6
II.3. Kaeden Peterson 6
II.4. Aryan Puthran 6
III. Project Requirements Specification **7**
III.1. Project Stakeholders 7
III.2. Use Cases 7
III.3. Functional Requirements 10
III.4. Non-Functional Requirements 14
III.5. Standards 15
IV. Software Design – From Solution Approach **19**
IV.1. Architecture Design 19
IV.2. Data Design 23
IV.3. User Interface Design 26
IV.4. Constraints 31
V. Test Case Specifications and Results **33**
V.1. Testing Overview 33
V.2. Environment Requirements 33
V.3. Test Results 34
VI. Projects and Tools Used **36**
VII. Description of Final Prototype **37**
VII.1. System Overview 37
VII.2. User Guide 38
VII.3. Major Use Cases 39
VII.4. Prototype Screenshots 40
VII.5. System Evolution 56
VIII. Product Delivery Status **58**
IX. Conclusions and Future Work **59**
IX.1. Limitations and Recommendations 59
IX.2. Future Work 59
X. Acknowledgements **60**
XI. Glossary **61**
XII. References **62**
XIII. Appendix **63**
A. Additional Use Cases 63

B. User Stories 63
C. Traceability Matrix 66

I. Introduction

I.1. Project Introduction

The Palouse Alliance Community Resource Platform is a digital initiative designed to make community health and wellness resources easier to find and easier to use across the Palouse region. Residents often need help quickly, but services are frequently scattered across separate nonprofit sites, county and state pages, flyers, and social media posts. That fragmentation creates real friction. People may not know what exists, which organization to contact, or whether information is still current. This platform addresses that gap by organizing trusted resources in one place and presenting them through a clear, searchable interface.

The Palouse Alliance for Healthy Individuals, Families, and Communities operates as a vital coalition of non-profits, healthcare providers, civic leaders, and volunteers. Despite their shared mission, coordinating cross-agency communication has historically relied on manual updates, resulting in silos of information. When an individual is in crisis or simply seeking routine support, navigating these silos can be overwhelming, leading to missed opportunities for intervention and assistance. A unified digital presence is not just a convenience; it is a critical infrastructure need for the region.

The platform is intended to support a wide range of users, including students, families, seniors, veterans, and individuals seeking housing support, food assistance, mental health services, or community programs. Rather than forcing residents to learn multiple websites and terminology, the platform will provide consistent categories, filters, and plain language descriptions. It will also highlight upcoming events and deadlines through a shared community calendar so residents can plan ahead instead of finding information at the last minute.

Accessibility is a core goal. The project will follow established accessibility guidance [1–3, 5] so that residents using assistive technologies, mobile devices, or limited bandwidth can still access critical information. In addition, the platform supports the organizations behind the services by reducing repeated outreach work and enabling simpler updates. Over time, this creates a more reliable community information hub, improves coordination among participating groups, and reduces the information gaps that can prevent people from getting help.

I.2. Background and Related Work

Currently, information about local services is spread across dozens of different websites and social media accounts, making it difficult for residents to find what they need. Many organizations are available to help, yet it is nearly impossible for residents to access the full range of services in a single search. This project is inspired by the Whitman County Veterans website (whitmancountyveterans.com), which brings together vital information for local veterans. Research on community resource platforms highlights the importance of good design and accessibility [1–4].

Previous attempts to bridge this communication gap have largely relied on decentralized social media groups, email listservs, or physical community bulletin boards. However, these methods suffer from significant drawbacks. Social media feeds are algorithmically driven, making chronological search and categorical filtering nearly impossible for users seeking specific, immediate help. Furthermore, these informal channels lack a verification process, occasionally leading to the spread of outdated or inaccurate service details. By contrast, a dedicated platform

ensures that all posted resources are vetted, categorized, and maintained by authorized community members.

Research into community informatics and digital resource platforms demonstrates that user adoption hinges on trust, data accuracy, and intuitive navigation [1–4]. The Whitman County Veterans website successfully addressed these elements by empowering a central organization to manage content updates, establishing a reliable "single source of truth." Our platform will adapt this proven model for a broader demographic, ensuring that the Palouse Alliance can seamlessly curate a much larger volume of community data while maintaining high standards of accuracy.

To deliver a custom-made platform featuring a user-friendly interface for searching, community calendars, and posting, a diverse technical skill set is required. As the project develops, these necessary skills will include modern full-stack web development, user authentication and data security, **SQL**, **AWS**/cloud infrastructure, UI/UX design, database design, and content moderation systems.

I.3. Project Overview

The Palouse Alliance Community Resource Platform will offer a community calendar and tools for organizations to post flyers for events, and meeting dates. Some core requirements include accessibility and responsiveness; the site should be available on laptop, tablet, and mobile devices, with support for those with disabilities. Members of the Alliance can keep their listing current and share announcements. Visitors will be able to filter to make it easier to connect with the relevant event hosts. The calendar shows upcoming deadlines and events, while admins can moderate what is being posted to ensure the content is accurate and appropriate for the community. Technical development will rely on modern web **HTML** frameworks [4]. By focusing on making the website easy to use for the public and members, the project aims to improve events and lower barriers for everyone.

The system is structured around three distinct user experiences: the general public, organization members, and system administrators. For the general public, the platform acts as a frictionless directory and calendar requiring no account creation to browse or search. For organization members, the system provides a secure portal to draft event listings, upload promotional flyers, and manage their agency's public profile. Finally, for administrators, the platform offers a robust moderation queue to review, approve, or reject submissions, ensuring that the platform remains free of spam and strictly aligned with the Alliance’s community guidelines.

Ultimately, we expect to see increased discoverability and accessibility, better coordination, and a scalable platform that can be extended or adapted to future needs. We also aim to significantly reduce the manpower required to cross-post events and meeting dates, ensuring Alliance members can spend less time managing digital overhead and more time focusing directly on the Palouse community.

II. Team Members & Bios

II.1. Omar Adil



Omar Adil is a Computer Science student at Washington State University, graduating in December 2026. As a SWE Intern at Silk, he designed a session management framework in Python for secure communication with Silk Virtual SAN platforms, and built an API construction engine for RESTful requests and authentication. Omar has experience developing reusable API helpers and automating system validation. Proficient in Python, C/C++, and C#. He is skilled with PostgreSQL, Node.js, and DevOps tools like Docker and GitHub. For the project, Omar will focus on backend and frontend integration.

II.2. Vijay Deva



Vijay Deva is a Computer Science major at Washington State University, expecting to graduate in December 2026. He previously interned at People Tech Group, where he developed and deployed a parking space detection machine learning pipeline using Yolo-NAS, achieving 97.7% accuracy, and integrated real-time model inference into a cloud-based management system. Vijay also built React and AngularJS UI components and implemented InfluxDB-powered data visualizations. He is adept with Python, C#, JavaScript frameworks, PostgreSQL, and Docker. For the project, Vijay will contribute to machine learning integration, full-stack development, and deployment of dashboards.

II.3. Kaeden Peterson



Kaeden Peterson is pursuing a Computer Science degree at Washington State University, class of December 2026. As an EECS TA, he supported courses by clarifying concepts, grading assignments, and ensuring consistent evaluation standards. Skilled in Python, C, C++, C#, and Java, Kaeden is also proficient with Docker, Linux/Bash, and collaborative tools like Git. His experience as a TA allows him to manage technical and team responsibilities efficiently. For the project, Kaeden will contribute to core backend development.

II.4. Aryan Puthran



Aryan Puthran is a Computer Science student at Washington State University, graduating December 2026. As a VCEA Teaching Assistant, he supported over 200 students across programming and software engineering courses, offering debugging help, grading assignments, and facilitating group discussions to reinforce key concepts. During his Data Engineering internship at T-Mobile, he developed Grafana dashboards, analyzed network and location data, automated visualizations, and migrated legacy automation scripts to Python to improve maintainability. Familiar with C++, Python, **SQL**, Grafana, and CI/CD tools, Aryan will take on the role of database manager and QA manager in the project.

III. Project Requirements Specification

The Project Requirements Specification section details the functional and non-functional requirements for this project. Functional requirements cover essential features like displaying events, membership, posting content, and searching. Non-functional requirements outline system constraints related to usability, reliability, branding, and maintainability. This section also includes **use cases**, which aligns requirements with objectives for development of the application.

III.1. Project Stakeholders

Allison Rockwell (allison.rockwell@wsu.edu) and Christine Oakley serve as the primary clients. They are responsible for communicating requirements, reviewing deliverables, and providing consolidated feedback from the Alliance membership. Beyond the clients, the main stakeholders include Alliance organizations, community members, event organizers, volunteers, and technical maintainers.

During the initial stakeholder meeting, the clients emphasized the need for a platform with a low learning curve for both end users and contributors. For Alliance members, the ability to easily update listings, share events, and post flyers using an integrated calendar is essential. Meanwhile, the general public requires reliable information, clear categorization, and consistent branding.

To successfully serve these groups, the platform must meet several core requirements [3–6]:

- Usability & Access: It must have a simple interface, be highly accessible, and remain fully responsive across laptop, tablet, and mobile devices.
- Security & Values: It must be secure, respect the privacy of organization members, and align with community values of inclusivity and support.
- Maintainability: It must be low maintenance and feature an intuitive content management interface to minimize technical overhead.

Communication between the development team and the Alliance will occur through regular biweekly meetings. These check-ins will be used to demonstrate progress, verify requirements, and gather iterative feedback.

III.2. Use Cases

This section describes typical scenarios in which users interact with the Palouse Alliance Community Resource Platform. **Use cases** clarify how actors achieve goals within the system and show the relationships between system requirements and the user experience [7]. See *Figure 1* below for an overview of system actors and their main actions.

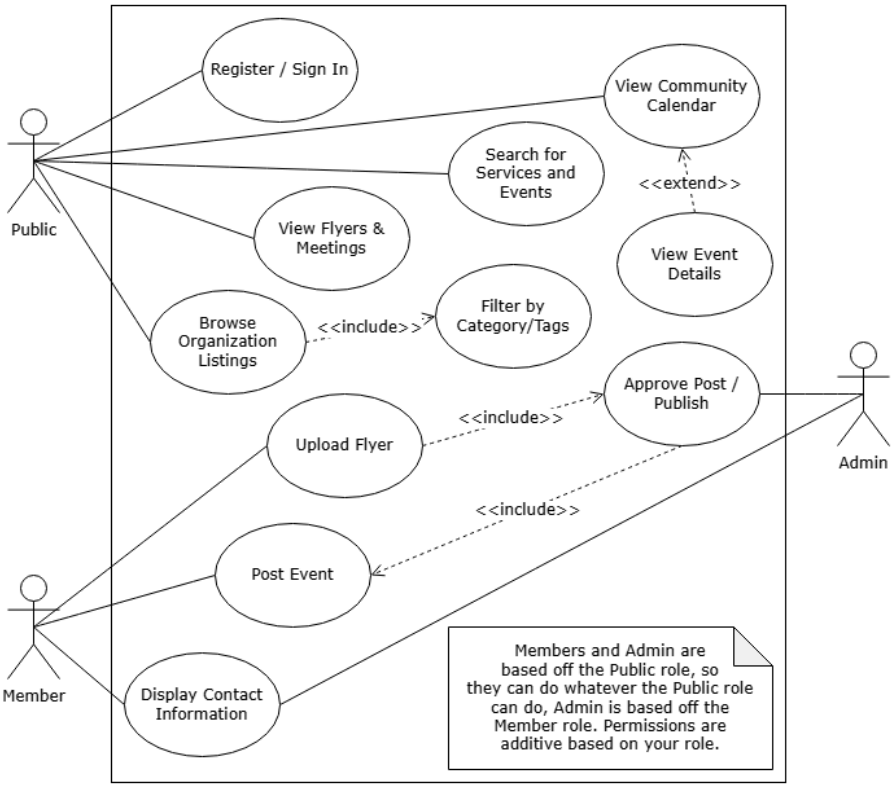


Figure 1: Use Case Diagram

III.2.1. [UC-1] Search and Attend Events

Use Case	Search and Attend Events
Actors	Public user, Member
Pre Condition	User is on the homepage (login not required).
Post Condition	User discovers events and may add to their calendar or explore details.
Main Flow	- User enters keywords in the search bar. - User optionally applies category or tag filters. - System displays a list of relevant services or events. - User selects an event to view more information. - User can save event details.
Alternative Flow	- No results found, user can adjust filters or submit a request for help.
Related Requirements	FR-1: Search for Services and Events FR-2: Category-Based Filtering FR-3: Event Calendar Integration FR-6: View Flyers and Meetings

III.2.2. [UC-2] Upload and Approve Event or Flyer

Use Case	Upload and Approve Event or Flyer
Actors	Member (uploads), Admin (approves)
Pre Condition	Member is logged in and accesses the event/flyer submission interface.
Post Condition	Event or flyer is posted to the calendar after admin approval.
Main Flow	- Member logs in and navigates to the submission page. - Member enters information and uploads files. - System sets item status as pending approval. - Admin logs in, reviews pending posts. - Admin approves (publishes) or rejects the submitted item.
Alternative Flow	- Admin rejects item, flyer/post isn't shown on public page.
Related Requirements	FR-4: Member and Admin Account Management FR-5: Event and Flyer Upload and Approval FR-16: Email Notifications for New Resources

III.2.3. [UC-3] Manage Organization Listing

Use Case	Manage Organization Listing
Actors	Member, Admin
Pre Condition	Member or admin is logged in and have access to their organization listing.
Post Condition	Organization listing is added, updated, or deleted.
Main Flow	- Member logs in and access the organization management page. - Member edits existing listings or create a new one (description, contacts, category, privacy). - Member saves changes. - System submits for admin approval.
Alternative Flow	- Admin removes listing, system archives it.
Related Requirements	FR-4: Member and Admin Account Management FR-7: Organization and Agency Listings FR-8: Display Contact Information FR-9: Display Contact Information

III.2.4. [UC-4] Volunteer Opportunity Discovery

Use Case	Volunteer Opportunity Discovery
Actors	Public user, Member
Pre Condition	User is on the homepage.
Post Condition	User discovers and explores volunteer opportunities via **GivePulse** integration.
Main Flow	- User navigates to the volunteer section. - System displays available opportunities via **GivePulse**. - User reviews details and may register or visit the external site.
Alternative Flow	- No current opportunities listed.
Related Requirements	FR-10: **GivePulse** Integration

III.2.5. [UC-5] Access Community Analytics

Use Case	Access Community Analytics
Actors	Admin
Pre Condition	Admin is logged in.
Post Condition	Admin views registration counts, attendance data, and other analytics.
Main Flow	- Admin logs into the dashboard. - Admin selects the analytics or reports section. - System displays up-to-date metrics and graphs.
Alternative Flow	- Data unavailable, system notifies admin of delay or error.
Related Requirements	FR-11: Analytics and Attendance Tracking

Please see the Appendix for additional **use cases**.

III.3. Functional Requirements

Functional requirements define the specific features, actions, and capabilities the platform must provide to its users. These requirements describe how users interact with the system and what tasks they can accomplish.

III.3.1. Search and Discovery

Functional Requirement	[FR-1] Search for Services and Events
Description	The system will provide a search bar enabling users to input keywords and find relevant services or events.
Source	Initial functional requirements, meeting notes.
Priority	Level 0 (Essential functionality)
Functional Requirement	[FR-2] Category-Based Filtering
Description	The system will allow users to filter services or events by selecting categories (e.g. Housing, Health) and/or by event-specific tags.
Source	Initial functional requirements, meeting notes.
Priority	Level 0 (Essential functionality)

III.3.2. Events and Calendar

Functional Requirement	[FR-3] Event Calendar Integration
Description	The platform will provide a public calendar which displays all approved events. Events can be added or edited (posting/editing only by members; approval required before publication).
Source	Initial functional requirements, meeting notes.
Priority	Level 0 (Essential functionality)
Functional Requirement	[FR-6] View Flyers and Meetings
Description	Users (including public) can view flyers, events, and upcoming meetings from the calendar.
Source	Initial functional requirements, meeting notes.
Priority	Level 0 (Essential functionality)
Functional Requirement	[FR-17] Event Lifecycle and Data Retention
Description	The system will automatically purge event records and associated media older than one month from the **Supabase** database to conserve resources. Public queries will be restricted to displaying events occurring within three

	months of the current date.
Source	Sprint 4 planning notes, meeting notes.
Priority	Level 0 (Essential functionality)

III.3.3. User Management

Functional Requirement	[FR-4] Member and Admin Account Management
Description	The system will support membership registration (“click here to become a member”) and sign in abilities. Member and admin accounts will be verified upon creation.
Source	Meeting notes.
Priority	Level 0 (Essential functionality)

III.3.4. Content Management

Functional Requirement	[FR-5] Event and Flyer Upload and Approval
Description	Members can upload events and flyers (as easy as sending an email). All posts require an admin’s approval before becoming public.
Source	Meeting notes.
Priority	Level 0 (Essential functionality)
Functional Requirement	[FR-7] Organization and Agency Listings
Description	The system will include a dedicated page to listing community agencies/members.
Source	Meeting notes.
Priority	Level 1 (Desirable functionality)

III.3.5. Information Display and Privacy

Functional Requirement	[FR-8] Display Contact Information
Description	Listings in the calendar will show relevant contact information, available to all users (including public).
Source	Initial functional requirements.

Priority	Level 0 (Essential functionality)
Functional Requirement	[FR-9] Selective Display of Personal Information
Description	The platform will allow members to choose which personal information to share with optional fields and a disclaimer during registration and posting.
Source	Meeting notes.
Priority	Level 1 (Desirable functionality)

III.3.6. Volunteer and Community Features

Functional Requirement	[FR-10] **GivePulse** Integration
Description	The site will include a dedicated section linking **GivePulse** to improve volunteer activities.
Source	Meeting notes.
Priority	Level 0 (Essential functionality)

III.3.7. Platform Support and Analytics

Functional Requirement	[FR-11] Analytics and Attendance Tracking
Description	The system will track and display basic analytics (e.g. registration counts, attendance), available to admin users. Note that analytics tracking was formally deprioritized per client preference to prevent redundant features.
Source	Meeting notes.
Priority	Level 2 (Optional functionality)
Functional Requirement	[FR-12] Custom Domain Name Support
Description	The platform will support deployment under a custom domain name. Elevate priority from Level 1 to Level 0 (Essential functionality) to reflect the formal Sprint 5 commitment migrating from Vercel staging to the client's official custom domain.
Source	Meeting notes.
Priority	Level 0 (Essential functionality)
Functional	[FR-13] Branding and Theming

Requirement	
Description	The website will use logos, palettes, and branding guidelines provided by the stakeholders.
Source	Meeting notes.
Priority	Level 1 (Desirable functionality)
Functional Requirement	[FR-14] Responsive Web Design
Description	The platform will be responsive for laptop, tablet, and mobile displays.
Source	Meeting notes.
Priority	Level 0 (Essential functionality)
Functional Requirement	[FR-15] Search Engine Display
Description	Website pages will be visible in web search engines.
Source	Meeting notes.
Priority	Level 2 (Optional functionality)
Functional Requirement	[FR-16] Email Notifications for New Resources
Description	The system will send an email notification to all members when a new resource is posted on the website.
Source	Email from Chris.
Priority	Level 0 (Essential functionality)
Functional Requirement	[FR-18] FAQ Page / Knowledge Base
Description	The platform will provide a responsive, publicly accessible FAQ section or page populated with approved answers to common community inquiries.
Source	Client consultation on 09/09/2026.
Priority	Level 1 (Desirable functionality)

III.4. Non-Functional Requirements

Non-functional requirements focus on the qualities and operating standards of the platform rather than its direct functionalities. They specify how the system performs under certain conditions or constraints and help ensure a quality user experience.

Non-Functional Requirement	Description
[NFR-1] Usability	The interface should be intuitive and accessible for all users [3, 11]. Uploading content (events, flyers, etc.) should be as easy as sending an email. Accessibility standards are followed [1, 6].
[NFR-2] Scalability	The platform must handle a growing number of users and events without a degradation in performance [4].
[NFR-3] Reliability	The platform should be available 99% of the time, with minimal planned downtime.
[NFR-4] Maintainability	The tech stack should be modular and documented. Code reviews and version control will be used.
[NFR-5] Security and Privacy	Sensitive user data will be stored securely [13, 14]. Members have control over public information [15]. Authentication will be used.
[NFR-6] Platform Compatibility	The system will be accessible on modern browsers and devices (laptops, tablets, and phones).
[NFR-7] Brand Consistency	The site will consistently apply provided branding, including logos and palettes.

III.5. Standards

This section lists the technical, legal, and professional standards that inform our requirements and design choices. Each standard is mapped to the parts of the specification it affects, including functional requirements, non-functional requirements, design, testing, security, and accessibility.

III.5.1. Standards Table

Standard / Code	Domain	Description	Application in Project
IEEE 830-1998: Software Requirements Specification [7]	Software Documentation	Guidance on the structure and content of SRS documents.	Used to organize functional and non-functional requirements, **use cases**, user stories, and the traceability matrix. Helps ensure requirements are clear, testable, and verifiable across the SRS.
ISO/IEC/IEEE 12207:2017: Software Life Cycle Processes	Project Management / Process	Defines **lifecycle** stages and roles.	Shapes project phases from requirements through maintenance, and supports change control in System

Standard / Code	Domain	Description	Application in Project
[8]			Evolution.
IEEE 1016-2009: Software Design Description (SDD) [9]	Design Documentation	Structure for describing software architecture.	Guides design artifacts such as module descriptions and interfaces. Supports maintainability (NFR-4) and documentation of calendar, listing, authentication, and API components.
IEEE 829-2008: Test Documentation [10]	QA / Testing	Formats for test plans, cases, and reports.	Used to document verification and validation for search, posting and approval flows, calendar features, accessibility, and regression testing. Mapped from FR-1 to FR-6, and FR-14 and FR-16.
W3C **WCAG** 2.2 (2023): Web Content Accessibility Guidelines [11]	Web / UI Accessibility	Success criteria and techniques to make web content perceivable, operable, and understandable.	Applied to UI, forms, calendars, flyers, and listing pages. Covers text alternatives, keyboard navigation, and clear labels and error messages. Supports NFR-1 and FR-3, FR-6, and FR-14.
WAI-ARIA 1.2 (2021): Accessible Rich Internet Applications [12]	Web Accessibility	Properties for accessible content.	Used for interactive components such as calendar widgets, filters, dialogs, and status messages to improve screen reader and keyboard support. Supports NFR-1, FR-2, and FR-3.
OWASP ASVS v4.0.3 (2021): Application Security Verification Standard [13]	Security	Requirements for authentication, sessions, and input validation.	Applied to member and admin accounts, session handling, upload validation, CSRF protections, and secure logging. Supports FR-4, FR-5, FR-16, and NFR-5.
IETF RFC 8446 (2018): TLS 1.3 [14]	Security / Transport	Transport protocol for encrypting data.	Required for all network traffic over **HTTPS**, including authentication, uploads, admin dashboards, and API calls. Supports NFR-5.
GDPR: Regulation (EU)	Data Protection /	Framework for processing personal	Informs data minimization, consent for contact details and

Standard / Code	Domain	Description	Application in Project
2016/679 (enforced 2018) [15]	Privacy	data and their rights.	notifications, and ability to access and delete. Supports FR-4, FR-9, FR-16, and NFR-5.
CAN-SPAM Act (2003) [16]	Email / Legal	U.S. requirements for commercial email and opt-out.	Applied to FR-16 email notifications, including accurate headers, clear identification, and unsubscribe or opt-out options.

III.5.2. How These Standards are Applied

Requirements and Lifecycle

- Requirements, **use cases**, and the traceability matrix follow IEEE 830 [7] so each requirement is clear, and testable and traceable.
- The overall project **lifecycle** follows ISO/IEC/IEEE 12207 [8], including roles, deliverables, and change management.
- Architecture and module documentation follow IEEE 1016 [9] to support long-term maintainability (NFR-4).
- Test plans and test artifacts follow IEEE 829 [10] to keep validation and regression testing consistent and auditable.
- Iterative stakeholder feedback loops are integrated into the lifecycle phases as defined by ISO/IEC/IEEE 12207 [8] to ensure continuous alignment with Alliance goals throughout development.
- Component interfaces and database schemas are explicitly detailed following IEEE 1016 [9] to facilitate a seamless operational handoff to future community technical maintainers.
- Defect tracking and resolution reports generated during User Acceptance Testing (UAT) will strictly adhere to the documentation formats prescribed by IEEE 829 [10] to ensure all software bugs are verifiably resolved before launch.

Accessibility

- Core user flows like search, filtering, calendars, flyer views, and submission forms will meet **WCAG** 2.2 [11] success criteria.
- Interactive UI elements will use WAI-ARIA [12] patterns where appropriate to ensure strong screen reader and keyboard support.
- U.S. accessibility guidance from Section 508 [1] and the ADA [6] will inform content clarity and effective communication.
- Image uploads, specifically event promotional flyers, will programmatically require alternative text (alt text) to comply with Section 508 [1] and **WCAG** [11] guidelines for assistive technologies.

Security and Privacy

- Authentication, authorization, session controls, and input validation will align with OWASP ASVS [13] at a level appropriate for a public community platform (NFR-5).

- All traffic will be encrypted using **HTTPS**, with a preference for TLS 1.3 as defined in RFC 8446 [14].
- Personal data handling will follow privacy-by-default practices inspired by GDPR [15], including consent and data minimization. Email notifications will follow CAN-SPAM rules [16].
- Controls for CSRF, XSS, and file upload validation will be implemented and captured in the security design.

Web Standards and SEO

- Semantic **HTML**, appropriate meta tags, and structured data will support accessibility and discoverability (FR-15).
- Branding and theming (FR-13) will be applied consistently without breaking semantic structure or assistive technology support.
- The platform's DOM structure will adhere strictly to W3C Web Standards [4] to ensure consistent rendering, responsive behavior, and forward compatibility across all modern desktop and mobile browsers.

III.5.3. Scope and Compliance

The platform aggregates community resources and events. It does not collect, store, or transmit Protected Health Information (PHI). Users cannot book medical appointments, submit health intake forms, or communicate securely with medical professionals through the website. Therefore, the Health Insurance Portability and Accountability Act (HIPAA) compliance standards remain entirely out of scope. If future updates require medical data handling, the architecture will require a complete security audit and the integration of HIPAA compliant infrastructure.

To maintain these strict data boundaries, the platform features clear disclaimers across all user input forms. The interface explicitly warns members against entering sensitive personal details or medical histories into open text fields. System administrators will actively reject event flyers or descriptions that attempt to solicit protected information from the public.

The system functions strictly as an informational directory and event calendar. The platform does not process financial transactions, accept monetary donations directly, or store credit card information. All volunteer registrations and event ticket purchases route through external third party platforms like GivePulse or dedicated partner websites. As a result, the Payment Card Industry Data Security Standard (PCI DSS) is completely out of scope for this project.

The platform serves a highly localized demographic within the Palouse region. Strict international or out of state data frameworks like the General Data Protection Regulation (GDPR) or the California Consumer Privacy Act (CCPA) do not legally apply to this specific jurisdiction and scale. The project still utilizes a risk based approach to user privacy by implementing data minimization. The system limits data collection to basic email addresses for authentication and avoids invasive third party tracking cookies.

IV. Software Design – From Solution Approach

IV.1. Architecture Design

This section describes the high-level technical structure of the platform, including the rationale for architectural decisions, the major layers and subsystems, and how system responsibilities are divided to have maintainable and scalable development.

IV.1.1. Overview

The Palouse Alliance Community Resource Platform is designed as a web application that prioritizes accessibility and ease of use. The system consists of a responsive frontend, a RESTful backend, and a relational database, combined with integration points for third-party services (i.e. **GivePulse** for volunteer opportunities). It follows a layered architectural pattern that separates presentation, logic, data management, and authentication, supporting maintainability and security.

The system utilizes a client-server architecture with layered decomposition. The reason for this approach is that it allows for a separation of concerns (frontend is isolated from backend logic and data management), making it easier to scale, maintain, and update the system independently. The REST API further ensures that new interfaces or integration points can be added without disrupting the backend layer.

IV.1.2. Subsystem Decomposition

Explains how the system is divided into major subsystems, each with distinct responsibilities such as search, calendar management, user authentication, content posting, analytics, and volunteer integration. For each subsystem, the interface and main functionalities are described.

IV.1.2.1. Search and Discovery Subsystem

a) Description: Handles user queries for searching services, events, and flyers. Supports keyword search, and category and tag filtering.
b) Concepts and Algorithms Generated:
    i) Keyword matching using **SQL** LIKE or full text search.
    ii) Category/tag filters applied as **SQL** joins or query modifiers.
c) Interface Description: Enables users to find relevant services, events, and flyers using keyword search, filters, and tags. Handles query logic and delivers results efficiently.

Services Provided

Service Name	Provided To	Description
SearchEventsAndServices	Frontend	Receives query, filter criteria, and returns matching queries.
ListCategoriesAndTags	Frontend	Filter UI population.

Services Required

Service	Service Provider
Access to events, organizations, and tags.	**Supabase**
Authentication if searching restricted or member only resources.	**Supabase**

IV.1.2.2. Events and Calendar Subsystem

a) Description: Manages the public calendar, event creation/submission, and flyer display. Handles posting/editing by members and moderation by admins.
b) Concepts and Algorithms Generated:
    i) **CRUD** operations for events and flyers.
    ii) Status field for moderation workflow (pending, approved, rejected).
    iii) Calendar rendering (frontend), date/time formatting.
c) Interface Description: Manages event listings, flyers, and the public calendar. Supports posting, editing, moderation, and viewing of events, ensuring timely access to community information.

Services Provided

Service Name	Provided To	Description
ListCalendarEvents	Frontend, public users	Returns events and flyers.
SubmitEventOrFlyer	Members	Creates pending events and flyers.
ApproveEventOrFlyer	Admin dashboard	Updates events and flyer status.

Services Required

Service	Service Provider
Authentication for submission/approval.	**Supabase**
Persistence for storing event/flyer details.	**Supabase**

IV.1.2.3. User Management Subsystem

a) Description: Handles member and admin registration, login, and account verification. Responsible for authentication, session management, and account profile privacy controls.
b) Concepts and Algorithms Generated:
    i) Secure password storage.
    ii) Role-based access (member, admin).

c) Interface Description: Controls member and admin registration, authentication, role assignment, and privacy options for account profiles and contact details.

Services Provided

Service Name	Provided To	Description
RegisterUser	Registration page	Creates user account.
LoginUser	Login page	Returns access token.

Services Required

Service	Service Provider
Database persistence for user records.	**Supabase**

IV.1.2.4. Content Management Subsystem

a) Description: Enables members to post new events and flyers, and allows admins to approve or reject submissions. Supports organization/agency listings management.
b) Concepts and Algorithms Generated:
   i) Submission workflows: file upload, status management.
   ii) **CRUD** operations for organization listings.
   iii) Approval workflow triggers email notification on publishing.
c) Interface Description: Allows organizational members to post new events, flyers, and listings; features workflows for moderation, editing, and approval.

Services Provided

Service Name	Provided To	Description
SubmitContent	Members	Allows posting new events, flyers, or organization listings.
ApproveContent	Admin interface	For moderation.
UpdateListing	Members and admins	Allows them to update organizational info.

Services Required

Service	Service Provider
Database storage for events, flyers, organizations.	**Supabase**
File upload service.	**Supabase**

Service	Service Provider
Email notifications for status updates (requirement FR-16).	N/A

IV.1.2.5. Volunteer & Analytics Subsystem

a) Description: Integrates **GivePulse** for volunteer opportunities. Provides admin access to basic analytics.
b) Concepts and Algorithms Generated:
    i) External API calls to **GivePulse**.
    ii) Basic registration and attendance counters.
    iii) Analytics dashboard rendering.
c) Interface Description: Integrates external volunteer services (like **GivePulse**), presents analytics dashboards for admin users, and tracks platform usage and attendance.

Services Provided

Service Name	Provided To	Description
ShowVolunteerOpportunities	Public	Fetches and displays **GivePulse** data.
ShowAnalytics	Admin	Can view registration and attendance stats.

Services Required

Service	Service Provider
GivePulse API or embed access.	**GivePulse**
Database read for registration/activity counts.	**Supabase**

IV.1.2.6. Subsystem Services Overview

The diagram below maps the major subsystems of the platform and the specific services they provide and require. The frontend application operates as the central client interacting with strictly isolated backend modules. This modular structure ensures that search functions, calendar displays, and user authentication operate independently. Developers can safely modify individual components without disrupting the rest of the application.

The visualization also highlights how these internal modules connect to essential external dependencies. Most subsystems rely on **Supabase** to handle data persistence and secure user authentication. The volunteer module connects directly to the **GivePulse** platform to retrieve active community service opportunities. Mapping these required interfaces clarifies the exact data boundaries needed to maintain a secure and scalable architecture.

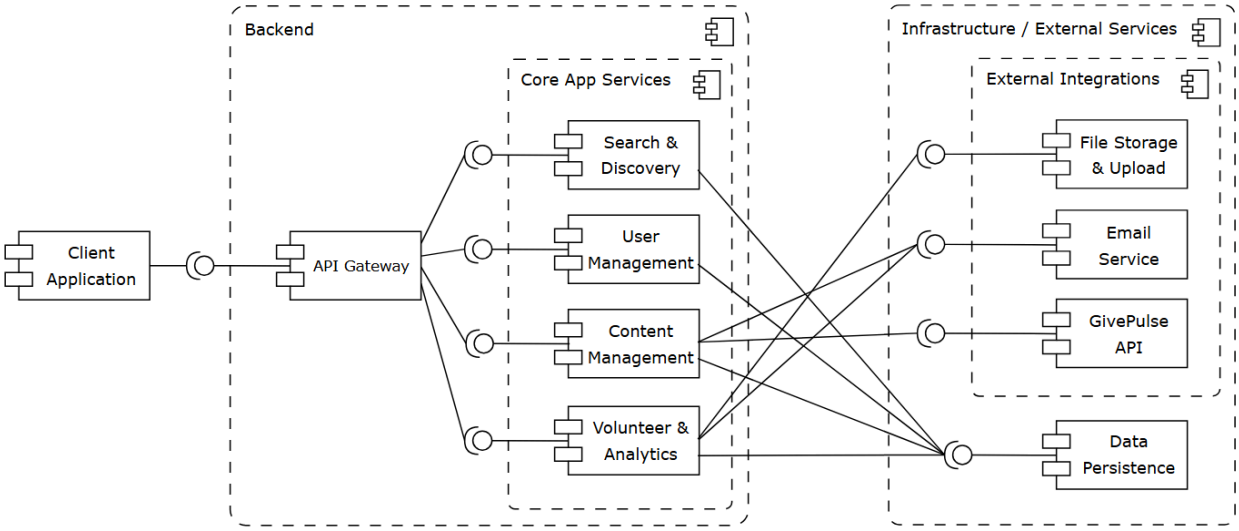


Figure 2: Ball and Socket Diagram

IV.2. Data Design

Details the structure of the platform's database, including tables, relationships, and the data entities that support the main features (users, organizations, events, tags, categories). Explains how data is stored and linked for reliability and performance.

IV.2.1. Internal Users Table (auth.users)

Stores authentication credentials for users of the platform. This table is managed by **Supabase** and handles secure storage of user email and password hashes. All platform users must have a corresponding record here.

Column	Type	Properties	Default Value
email	text	non-nullable	N/A
password	N/A	non-nullable	N/A

IV.2.2. Public Users Table (public.users)

Tracks individual users participating in the system, whether as members or admins. Links users to their affiliated organization and assigns their role within the platform.

Column	Type	Properties	Default Value
id	uuid	primary key, non-nullable	auth.uid()
role	user_role (“admin” or “member”)	non-nullable	member
organization_id	uuid	foreign key,	N/A

Column	Type	Properties	Default Value
		non-nullable	

IV.2.3. Organizations Table (public.organizations)

Represents organizations or agencies that participate in the platform, such as community groups or service providers. Contains descriptive details and relevant contact info.

Column	Type	Properties	Default Value
id	uuid	primary key, non-nullable	gen_random_uuid()
name	text	non-nullable	N/A
description	text	nullable	N/A
phone_number	text	nullable	N/A
email	text	nullable	N/A
location	text	nullable	N/A

IV.2.4. Events Table (public.events)

Contains all events and flyers posted by organizations, including scheduling, description, and approval tracking. Events are tied to organizations, can have categories, and are associated with tags.

Column	Type	Properties	Default Value
id	uuid	primary key, non-nullable	gen_random_uuid()
organization_id	uuid	foreign key, non-nullable	N/A
category_id	uuid	foreign key, non-nullable	N/A
status	event_status (“pending”, “approved”, “rejected”)	non-nullable	pending
title	text	non-nullable	N/A
description	text	nullable	N/A

Column	Type	Properties	Default Value
start_datetime	timestampz	non-nullable	N/A
end_datetime	timestampz	non-nullable	N/A
location	text	nullable	N/A
volunteer_url	text	nullable	N/A
flyer_url	text	nullable	N/A
created_by	uuid	non-nullable	N/A

IV.2.5. Tags Table (public.tags)

Provides categorization tags for events, allowing users to filter and search by topic.

Column	Type	Properties	Default Value
id	uuid	primary key, non-nullable	gen_random_uuid()
name	text	non-nullable	N/A

IV.2.6. Event-Tags Table (public.event_tags)

Associates tags with events using a many-to-many relationship. Each entry connects an event to a tag, enabling flexible filtering and searching.

Column	Type	Properties	Default Value
event_id	uuid	composite key, foreign key, non-nullable	N/A
tag_id	uuid	composite key, foreign key, non-nullable	N/A

IV.2.7. Categories Table (public.categories)

Defines broad categories for organizations and events. Used for filtering and browsing.

Column	Type	Properties	Default Value
id	uuid	primary key, non-nullable	gen_random_uuid()

Column	Type	Properties	Default Value
name	text	non-nullable	N/A

IV.2.8. Entity-Relationship Diagram

Describes how the main entities relate to one another, outlining the connections between users, organizations, events, and tags for consistent, scalable data management [17].

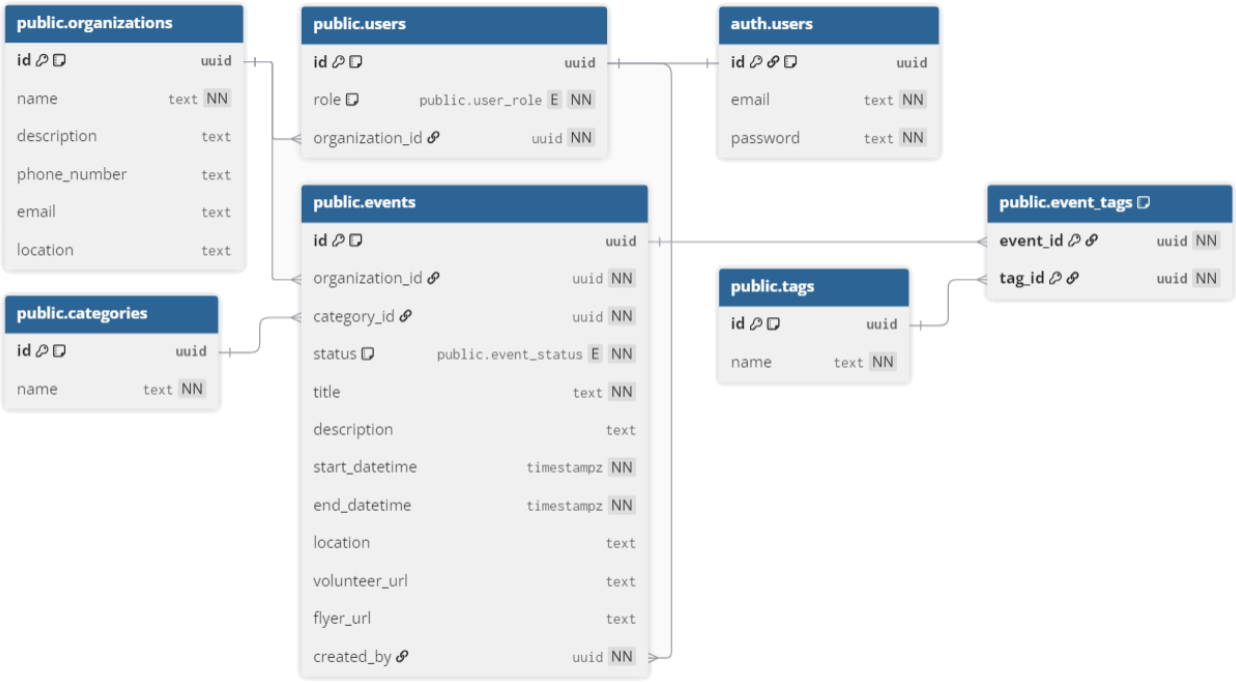


Figure 3: Entity Relationship Diagram

IV.3. User Interface Design

Outlines the approach and components used in the platform's interface, ensuring accessibility and ease of navigation for all device types. Describes core screens and accessibility standards.

IV.3.1. Home Page

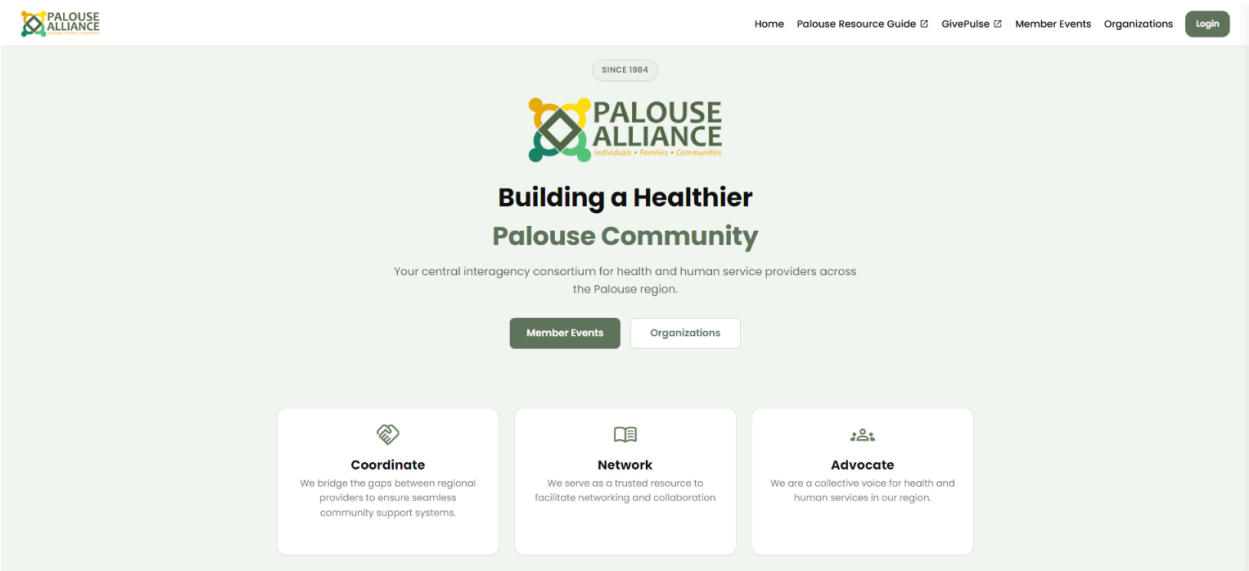


Figure 4: The home page.

Related Functional Requirements:

- FR-1: Search for Services and Events
- FR-2: Category-Based Filtering
- FR-6: View Flyers and Meetings
- FR-14: Responsive Web Design

IV.3.2. Login Page

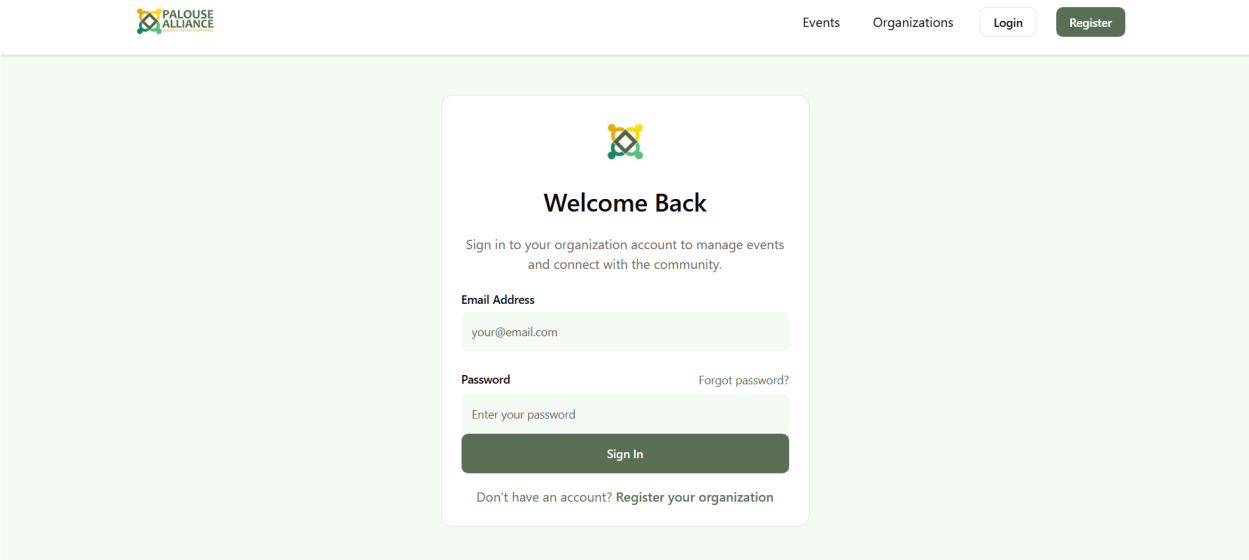


Figure 5: The login page.

Related Functional Requirements:

- FR-4: Member and Admin Account Management
- FR-14: Responsive Web Design

IV.3.3. Register Page

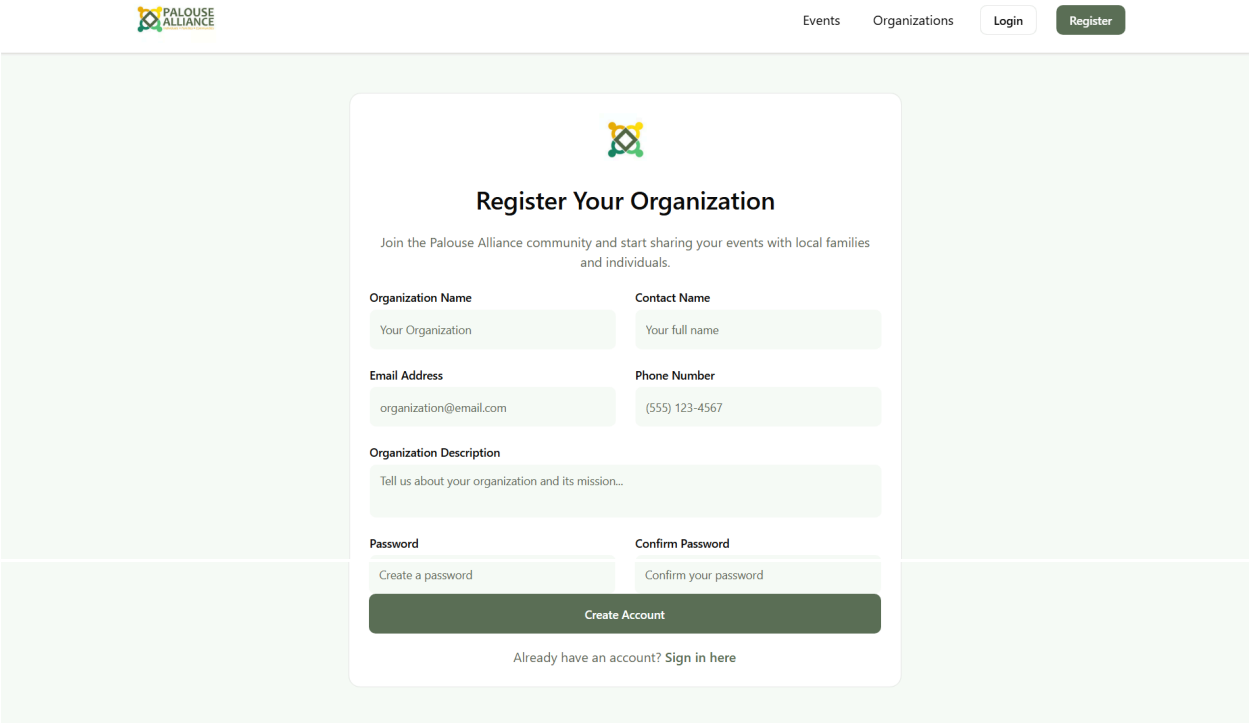


Figure 6: The register page.

Related Functional Requirements:

- FR-4: Member and Admin Account Management
- FR-14: Responsive Web Design

IV.3.4. Post Event Page

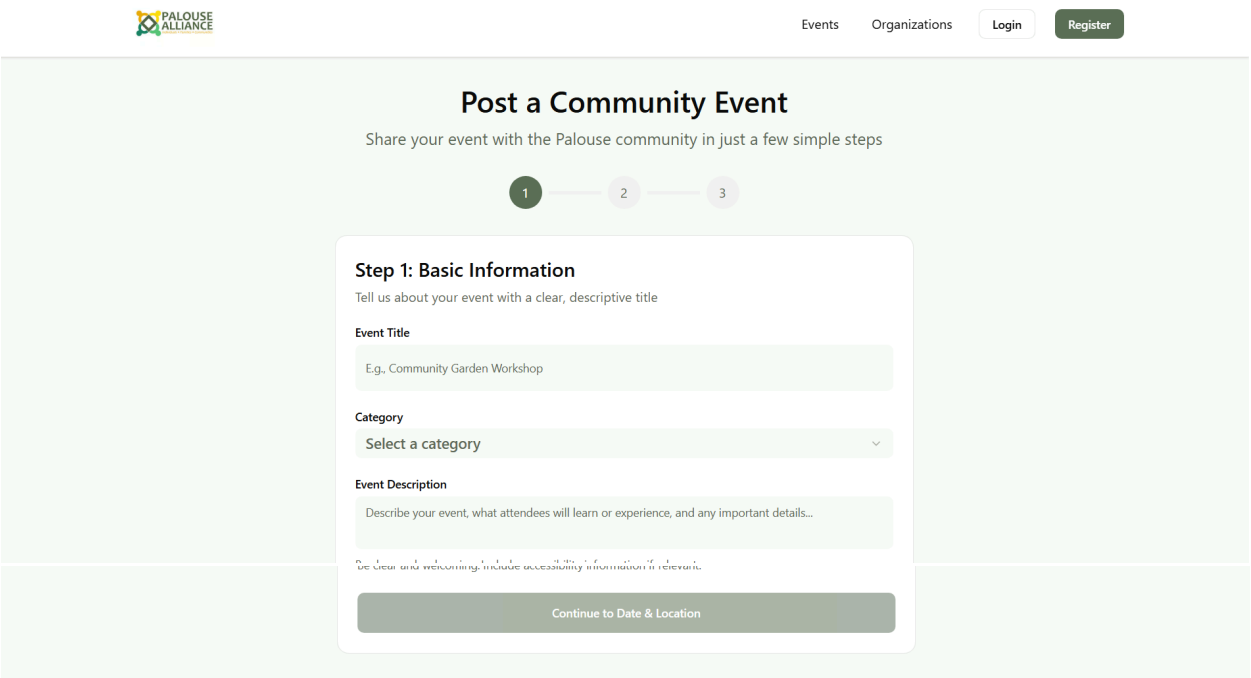


Figure 7: The post event page.

Related Functional Requirements:

- FR-5: Event and Flyer Upload and Approval
- FR-14: Responsive Web Design

IV.3.5. Dashboard Page

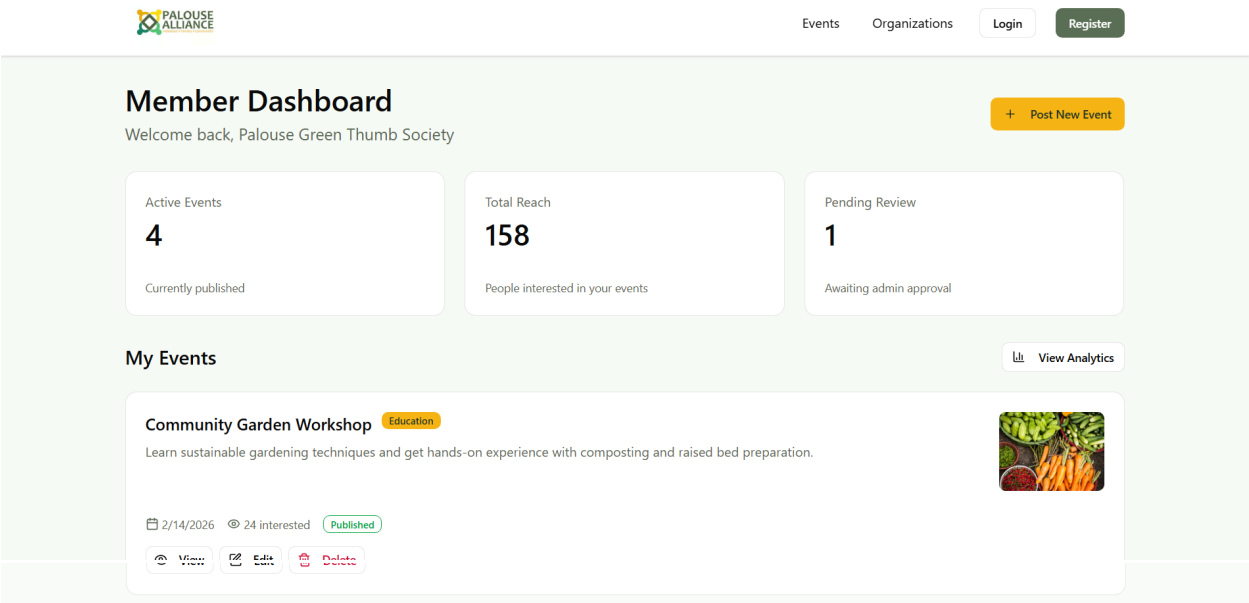


Figure 8: The dashboard page.

Related Functional Requirements:

- FR-6: View Flyers and Meetings
- FR-5: Event and Flyer Upload and Approval
- FR-11: Analytics and Attendance Tracking
- FR-14: Responsive Web Design

IV.3.6. Admin Page

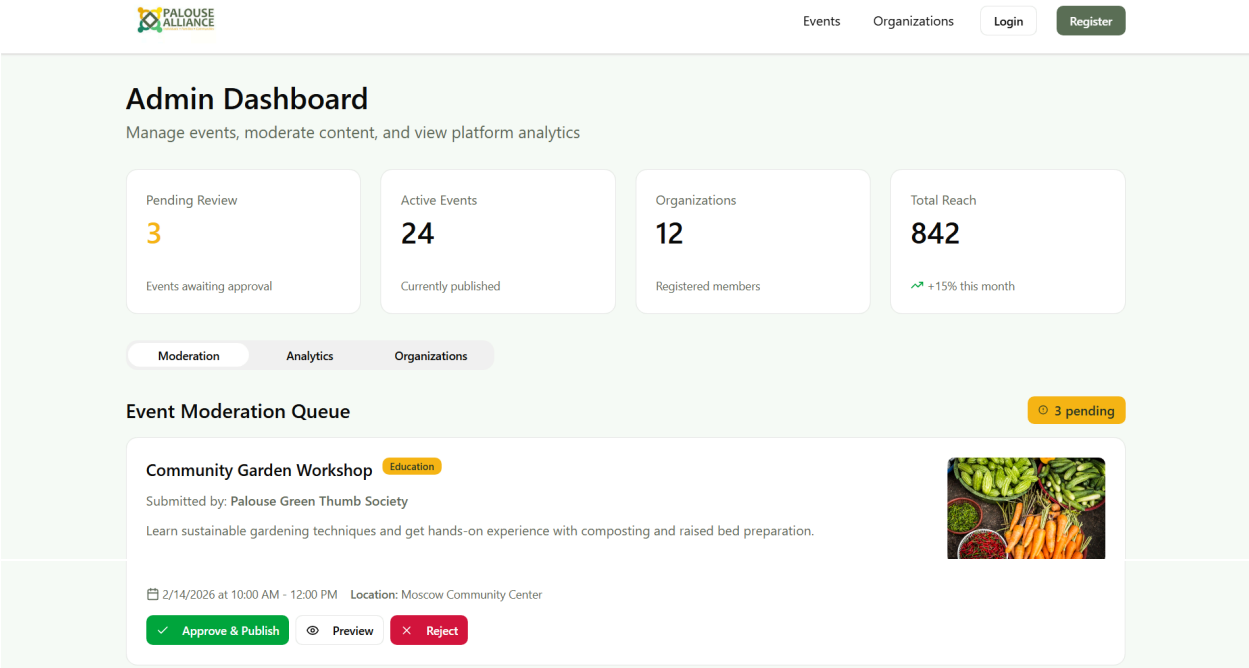


Figure 9: The admin page.

Related Functional Requirements:

- FR-4: Member and Admin Account Management
- FR-5: Event and Flyer Upload and Approval
- FR-11: Analytics and Attendance Tracking
- FR-16: Email Notifications for New Resources
- FR-14: Responsive Web Design

IV.3.7. Organizations Page

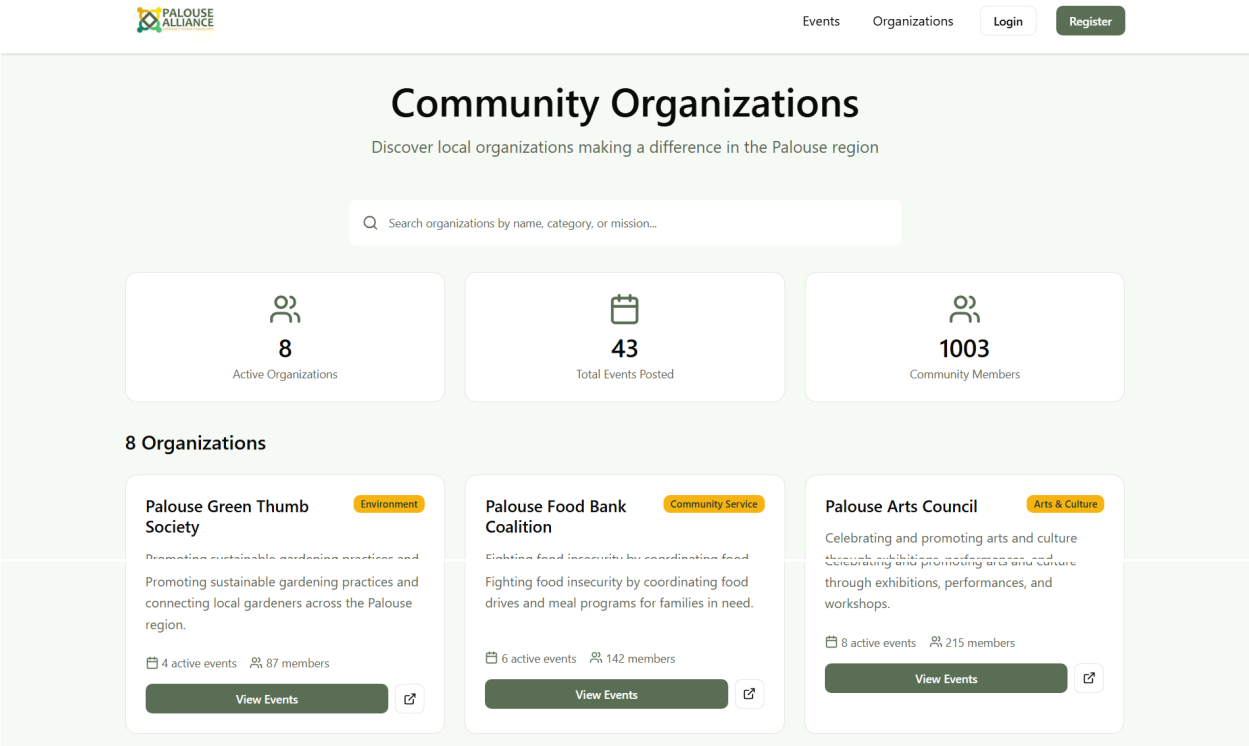


Figure 10: The organizations page.

Related Functional Requirements:

- FR-7: Organization and Agency Listings
- FR-9: Selective Display of Personal Information
- FR-14: Responsive Web Design

IV.4. Constraints

Lists the key limitations, requirements, and guidelines that shape the platform's development like accessibility, privacy, scalability, branding, and maintainability. Explains how these constraints affect technical and design decisions.

Category	Description	Impact
Accessibility	Must meet **WCAG** 2.2 and ADA guidelines for digital access.	Requires full keyboard navigation support and screen reader compatibility

Category	Description	Impact
		across all submission forms.
Security	Must enforce strict authentication and role-based access controls.	Only administrators can approve pending event submissions or modify organizational profiles.
Scalability	The system must support growing web traffic and a larger volume of daily event postings.	Cloud-hosted database architecture automatically handles increased user loads without requiring manual server upgrades.
Privacy	Must follow data minimization principles and protect member contact details.	Members use toggle settings during registration to decide which personal phone numbers remain visible to the public.
Brand Consistency	Must strictly adhere to Palouse Alliance visual identity guidelines.	Developers apply standardized color palettes and official coalition logos to all frontend interface components.
Maintainability	Codebase must remain highly organized and well-documented for future technical volunteers.	Modular component design allows new developers to add future features without breaking existing page layouts.
Performance	The platform must load quickly on limited mobile data networks and older devices.	Uploaded event flyer images undergo strict file size validation and compression to reduce bandwidth usage.
Data Storage	The system must manage database growth to minimize ongoing cloud hosting costs.	Automated database triggers permanently delete event records and associated media thirty days after completion.

V. Test Case Specifications and Results

V.1. Testing Overview

Testing for the Palouse Alliance Community Resource Platform verifies functional compliance (FR-1 through FR-16), core non-functional attributes, and overall system reliability prior to launch. A layered strategy balances automation in GitHub Actions with targeted manual verification and stakeholder acceptance.

V.1.1. Testing Levels

- Unit Testing: Component-level checks for React components, hooks, and helpers using Vitest and React Testing Library; backend logic and validation tested with mocked **Supabase** calls.
- Integration Testing: API endpoint and database interaction tests using Supertest, with UI flows evaluated via Playwright or Cypress against a containerized PostgreSQL instance or shared staging database.
- System & Functional Testing: End-to-end evaluation of core workflows, including event filtering, flyer submissions, admin approval queues, organization listing privacy controls, and GivePulse volunteer integrations.
- Standards & Non-Functional Verification: WCAG-aligned accessibility audits via axe and Lighthouse, performance profiling under normal workloads, and dependency vulnerability checks using Dependabot.
- User Acceptance Testing (**UAT**): Scenario-based validation with the primary client and community members to confirm readiness for launch.

V.1.2. Milestone Schedule

Phase	Timeline	Primary Objectives	Deliverables
Sprint 5	Oct 1 - Oct 31	Functional and validation testing, systematic defect logging, and closed beta execution.	Passing unit / integration suites, closed beta feedback with >5 community testers.
Sprint 6	Nov 1 - Dec 5	Extended **UAT**, defect remediation, client training, and operational handoff.	Validated end-to-end user flows, client sign off, and release readiness.

V.2. Environment Requirements

To support repeatable test execution within practical infrastructure constraints, testing operations are divided across three target environments.

V.2.1. Local Development Environment

- Runtime: Node.js with npm or yarn running the Vite/React application and API layers.

- Database Services: Docker and Docker Compose managing localized PostgreSQL containers for isolated persistence checks.
- Configuration: Parameterized local .env files for secrets management without committing credentials to version control.

V.2.2. Continuous Integration (GitHub Actions)

- Automated linting, dependency installation, and unit test execution on every pull request and push to the main branch.
- Ephemeral PostgreSQL containers initialized directly in CI jobs to execute integration suites against clean schemas.
- Credential injection handled through encrypted GitHub Secrets.

V.2.3. Shared Staging & UAT Environment

- Single cloud-hosted **Supabase**/PostgreSQL instance paired with a deployed web frontend served over HTTPS.
- Shared infrastructure allocated for end-to-end walkthroughs, system testing, and client UAT.
- Role-based test accounts (Public, Member, Admin) configured with synthetic, clearly marked seed data to avoid polluting demonstration assets.

V.3. Test Results

Test Category	Test Cases	Expected Result	Methodology / Tools
Unit Tests	Validate login form inputs; render Event Card component with missing flyer image.	Invalid email formats trigger inline UI error messages; component renders fallback default image without crashing.	Vitest, React Testing Library
Integration Tests	POST request to submit a new event; API call to fetch GivePulse volunteer data.	Database schema rejects unauthenticated POSTs; API returns 200 OK and correctly mapped JSON payload.	Supertest, Dockerized Postgres
System & Functional	End-to-end event submission workflow (FR-5); filter public calendar by “Housing” category (FR-2).	Member submits event, status becomes “Pending”, admin approves, and event appears on public calendar.	Playwright, Manual Checklists
Accessibility	Keyboard navigation	Users can tab	Lighthouse, axe,

Test Category	Test Cases	Expected Result	Methodology / Tools
	through the main menu; screen reader validation of event submission form fields.	through all interactive elements; Lighthouse and axe scores meet WCAG 2.2 AA thresholds.	Keyboard Nav
Security Hygiene	Attempt SQL injection on search bar; attempt to access `/admin` route without an admin session token.	**Supabase** sanitizes inputs preventing injection; unauthorized users are immediately redirected to the login page.	Dependabot, Code Review
Performance & Load	Simulate 100 concurrent users accessing the calendar; upload maximum file size (5MB) event flyer.	Page loads within 2.5 seconds under load; file uploads process without timing out or crashing the serverless function.	k6, Chrome DevTools
Cross-Device Compatibility	Render the application on iOS Safari, Android Chrome, and desktop Firefox; test mobile hamburger menu.	UI elements scale correctly without overlapping (FR-14); navigation functions seamlessly on touch screens.	BrowserStack, Chrome Device Toolbar
Data Lifecycle & Retention	Trigger the automated monthly database cleanup routine (FR-17) on synthetic expired data.	Event records older than 30 days are purged from `public.events` and associated storage buckets are cleared.	**Supabase** Cron/Triggers, Manual SQL
User Acceptance (**UAT**)	Client and beta testers navigate the system to create an organization profile and post a community flyer.	Testers successfully complete workflows without technical assistance and verify published listings appear correctly.	Client & Alliance Beta Testers

VI. Projects and Tools Used

The Palouse Alliance Community Resource Platform is a single-page application built on a modular web stack centered around Node.js 24.x, React 19, Vite, and **Supabase**. This architecture combines a responsive, component-based frontend with comprehensive cloud-hosted backend services, automated quality assurance, and streamlined continuous deployment.

Tool	Primary Purpose
React 19	Builds reusable UI components like event listing cards, calendars, and form modals.
Vite	Serves as the development build tool and bundler, providing hot module replacement and production asset optimization.
React Router DOM	Manages client-side single-page application (SPA) routing for discovery views, dashboards, and admin portals.
Supabase / PostgreSQL	Provides Backend-as-a-Service functionality and database storage for user roles, organization directories, and event records.
Supabase Auth	Handles secure, password-based user authentication and session management.
Supabase Storage	Hosts cloud buckets for community event flyers and media assets.
Vitest & React Testing Library	Executes automated unit and component-level tests to verify UI, form validation, and routing controls.
ESLint	Enforces code quality, style consistency, and development conventions across the codebase.
Lighthouse & axe	Conducts accessibility audits to ensure compliance with WCAG 2.2 and WAI-ARIA guidelines.
Git & GitHub	Organizes version control, code reviews, branch management, and feature tracking.
GitHub Actions	Automates continuous integration (CI) workflows on new pull requests.
Vercel	Hosts production and staging environments, managing client-side endpoints, route rewrites, and serverless functions.

VII. Description of Final Prototype

VII.1. System Overview

The Palouse Alliance Community Resource Platform is a responsive web application built with React 19, Vite, and React Router DOM. It provides a central hub where Palouse residents can find local health, wellness, and community support resources. The platform connects directly to **Supabase** through a centralized configuration file at `code/src/lib/supabase.js`. This backend integration manages user authentication, relational database tables for approved events and local organizations, and cloud storage for event flyers.

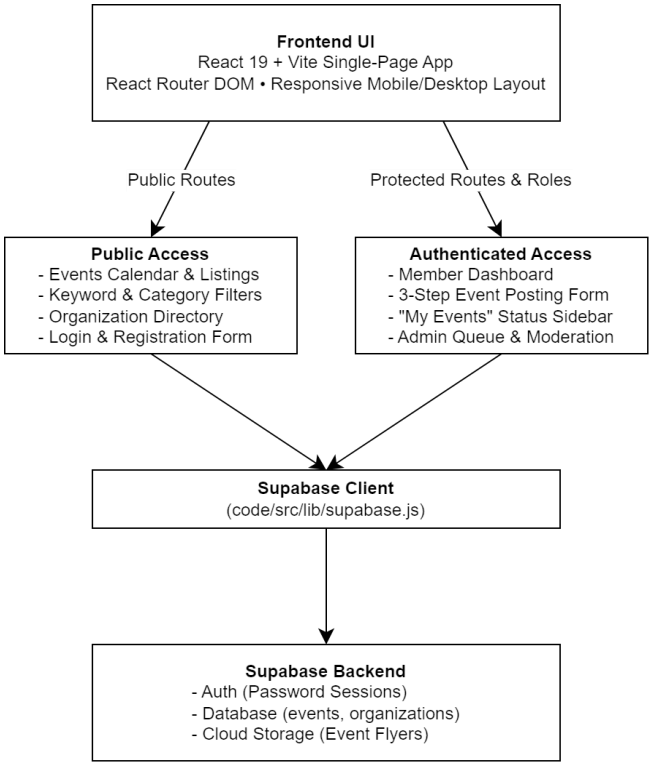


Figure 11: System overview showing the general structure of the platform..

The frontend uses client-side routing to split public features from authenticated member and administrative tools:

- Presentation Layer: The user interface relies on reusable modular components like Event Card, Event Calendar, and a responsive navigation bar.
- Routing Layer: React Router DOM manages transitions and blocks unauthorized access to member dashboards and administrative tools.
- Backend Integration Layer: The **Supabase** JavaScript client connects to database tables for real-time reads, writes, and authentication checks.
- Testing and Build Setup: The codebase runs on Node.js 24.x, uses Vitest and React Testing Library for automated component tests, uses ESLint for code quality, and deploys to Vercel via `code/vercel.json`.

VII.2. User Guide

VII.2.1. Public Visitor Workflows

- Discovering Events and Programs
    1. Open the landing page at `/` to view the master community calendar and featured activity cards.
    2. Switch between card listings and the interactive calendar view to examine scheduled dates.
    3. Enter keywords in the top search bar to narrow listings by program name or description.
    4. Select specific category or tag filters to isolate resources in areas such as housing support, food assistance, mental health services, or veteran outreach.
- Finding Community Organizations
    1. Navigate to the Organization Directory at `/organizations`.
    2. Search by agency name or filter entries by service category.
    3. Click any organization summary card to view service details, primary phone numbers, emails, physical locations, and verified contacts.
- Exploring Volunteer Opportunities
    1. Select the volunteer link in the main navigation.
    2. Review active community service postings retrieved through the GivePulse platform integration.
    3. Click an external posting link to register directly on GivePulse.
- Accessing Account Authentication
    1. Select Login in the top navigation bar to access the account portal at `/login`.
    2. Authenticated sessions grant verified members access to submission forms and personal dashboards.

VII.2.2. Authenticated Member Workflows

- Submitting an Event or Flyer
    1. Log in with verified member credentials and select Post Event from the navigation menu to open `/post-event`.
    2. Complete Step 1 by providing the formal event title and an explanatory description.
    3. Complete Step 2 by selecting appropriate service categories and topical tags.
    4. Complete Step 3 by entering the date, start time, end time, and physical or virtual location details.
    5. Complete Step 4 by uploading an optional event promotional flyer image.
    6. Submit the form to place the record directly into the moderation queue for administrator review.
- Tracking Personal Submissions
    1. Open the My Events panel from the member dashboard or site header.
    2. Monitor the real-time review status of each submission across pending, approved, and rejected states.

3. If an administrator rejects an event with feedback, view the entry within the panel to make required updates before resubmission.

VII.2.3. Administrative Workflows

- Moderating Event Submissions
    1. Log in with an administrator account and open the protected management suite at `/admin`.
    2. Open the moderation queue to inspect pending community submissions.
    3. Review the event timing, sponsoring organization, host contact information, and uploaded flyer graphics.
    4. Click Approve to publish the entry to the public calendar, update its status in the database, and trigger automated member notifications.
    5. Click Reject to withhold the event from public listings while keeping it accessible to the submitting member in their personal overview.
- Managing Platform Members and Organizations
    1. Open the user administration controls to register new user accounts, update profile roles, or deactivate credentials.
    2. Open the organization management directory to add new community partners or edit existing profiles, contact records, and service descriptions.

VII.3. Major Use Cases

Use Case ID	Primary Actor	Flow and System Interaction
UC-1: Search and Attend Events	Public Visitor	The visitor loads the main events route, submits search terms, applies category tags, and toggles between the card list and the visual calendar to inspect details.
UC-2: Upload and Approve Event or Flyer	Member and Administrator	A logged-in member completes the multi-step posting form and uploads an image. An administrator reviews the pending queue item, validates metadata, and issues an approval to publish the record to the public calendar.
UC-3: Manage Organization Listing	Member and Administrator	An administrator creates or updates partner directory records. Members can view their organizational details and submit update requests for administrative verification.
UC-4: Volunteer Opportunity Discovery	Public Visitor	The user navigates to the volunteer opportunities route, which displays current service openings integrated directly from GivePulse.
UC-5: Access Community Analytics	Administrator	The administrator opens the reporting interface to evaluate site traffic and event

Use Case ID	Primary Actor	Flow and System Interaction
		engagement. Note that this feature was formally deprioritized per client guidance to prevent redundant features.
UC-6: Manage Deployment and Branding	Administrator	An administrator updates custom domain settings, styling configurations, or search metadata to ensure the platform remains discoverable and aligned with Alliance branding.

VII.4. Prototype Screenshots

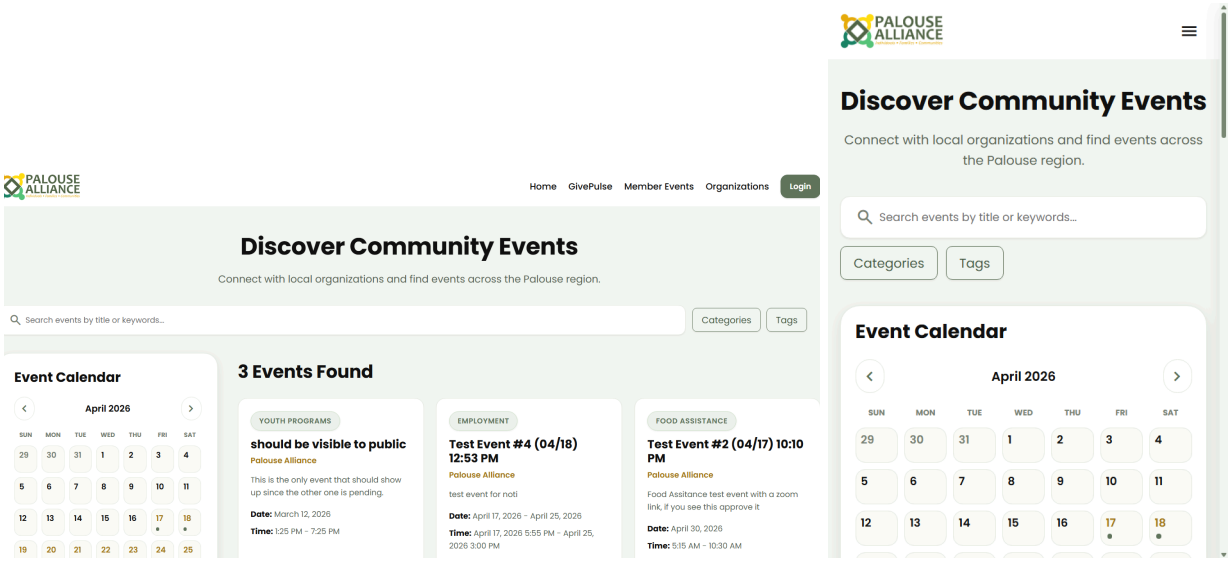


Figure 12: Events page layout displaying responsive design on both laptop and mobile viewports.

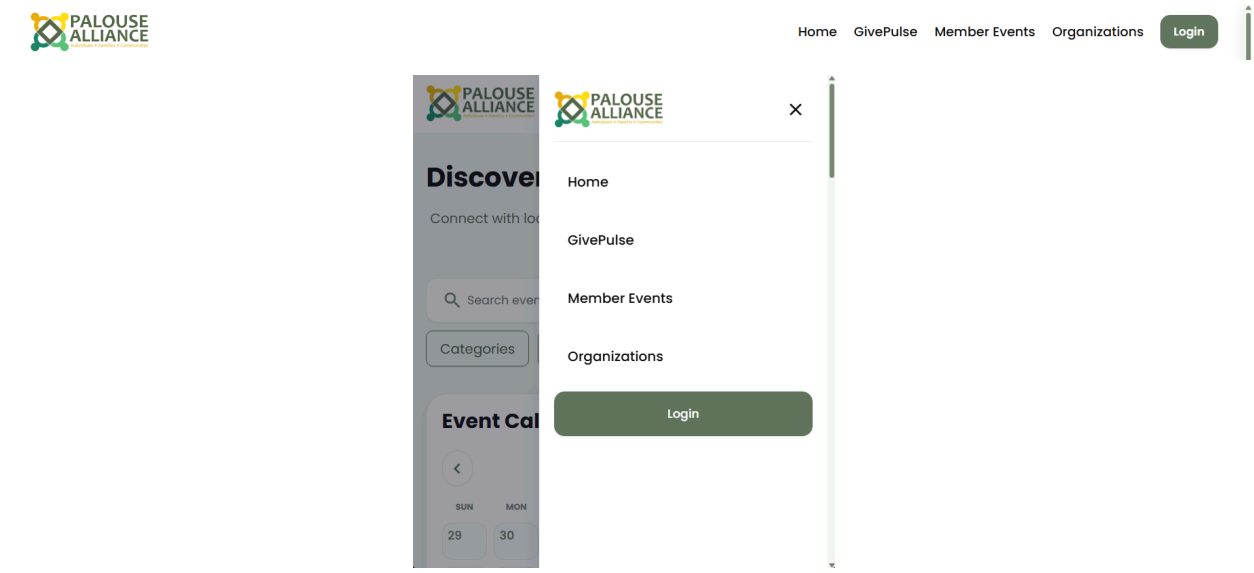


Figure 13: Global navigation bar showing responsive states across desktop and mobile screens.

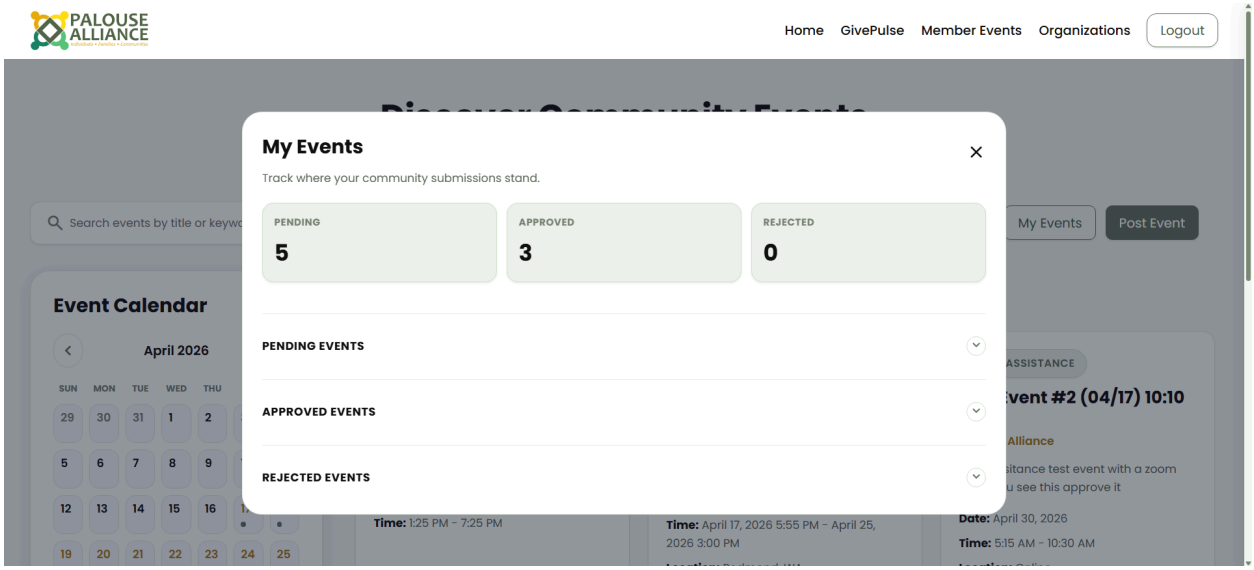


Figure 14: "My Events" pop-up interface accessed by an authenticated member or administrator.

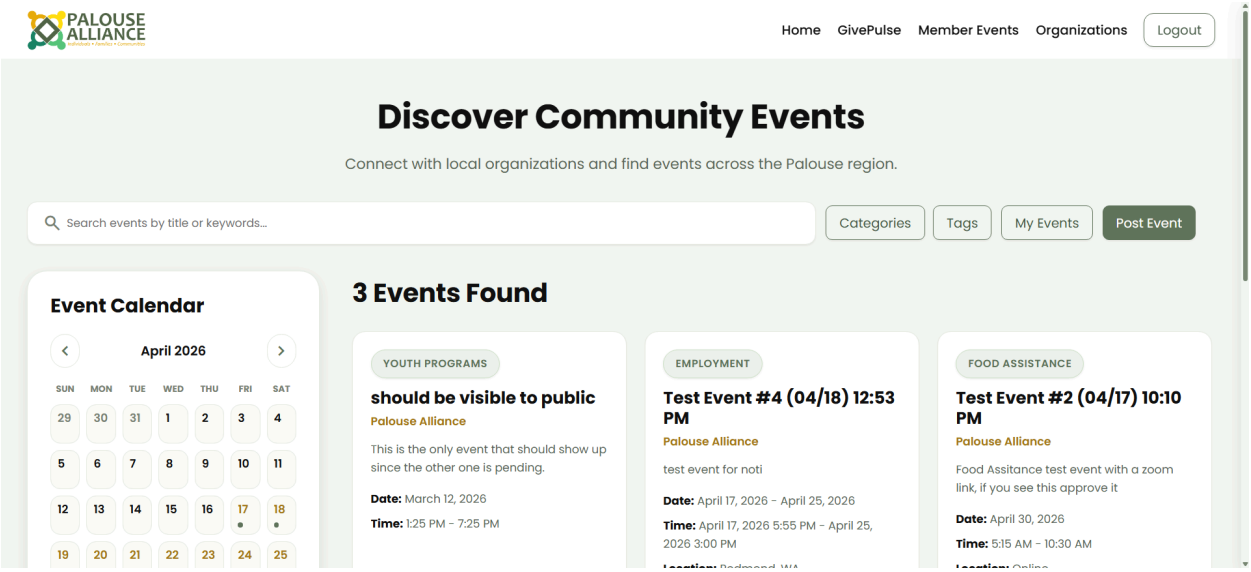


Figure 15: Community Events page for an authenticated user, featuring the activated "My Events" action button.

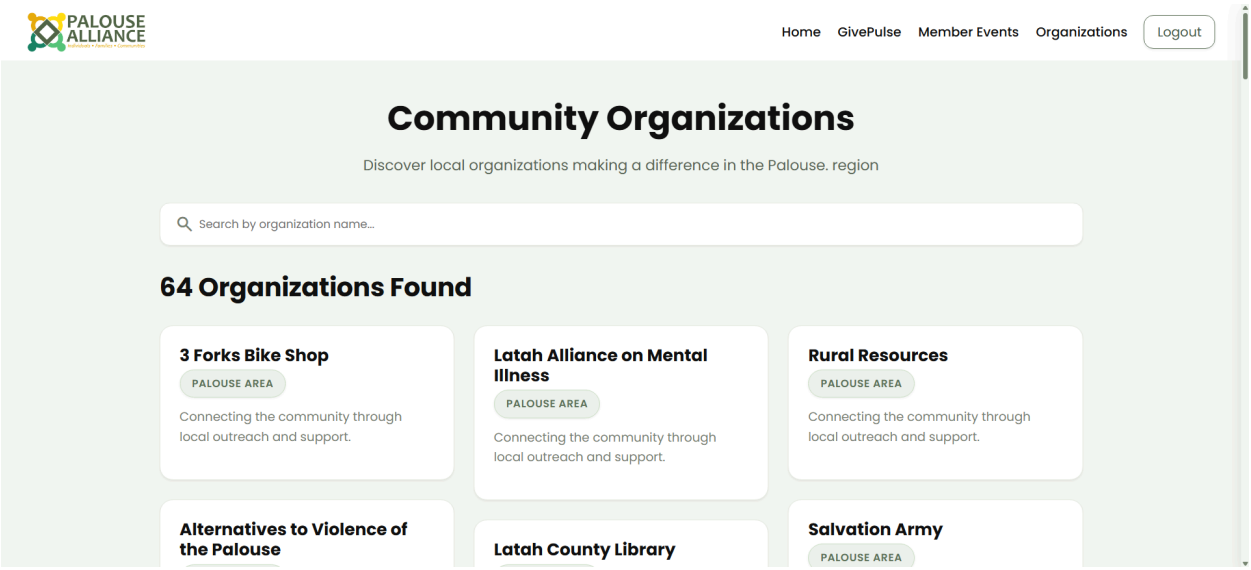


Figure 16: Public Organization Directory search view and contact summary cards.

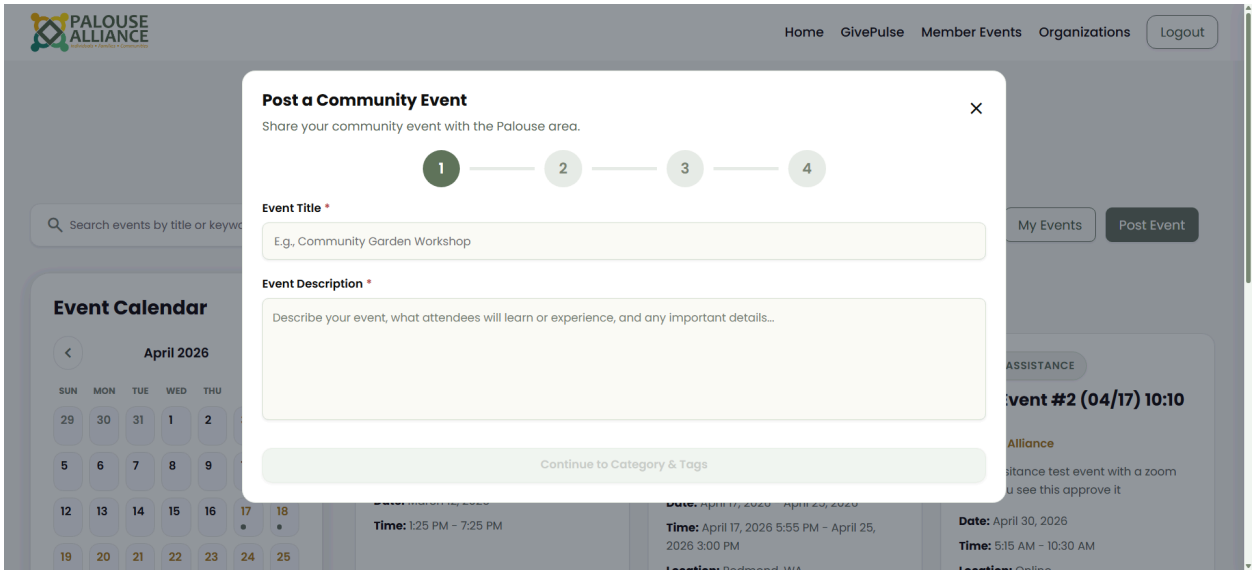


Figure 17: Multi-step event submission modal accessible to authenticated users.

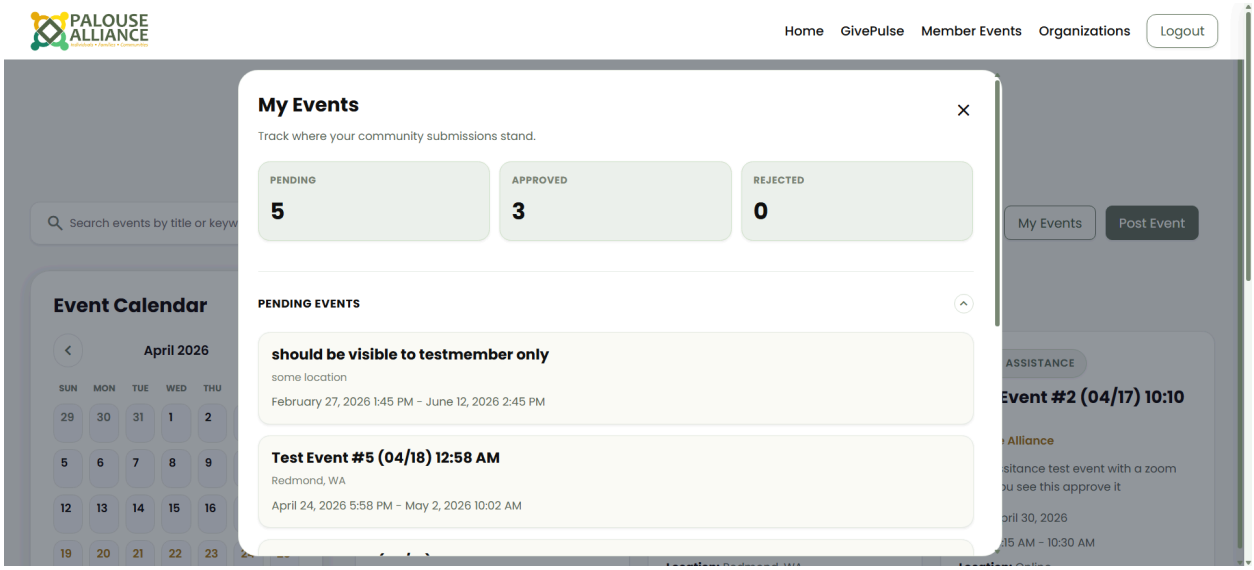


Figure 18: "My Events" drawer/pop-up showing submitted listings post-creation.

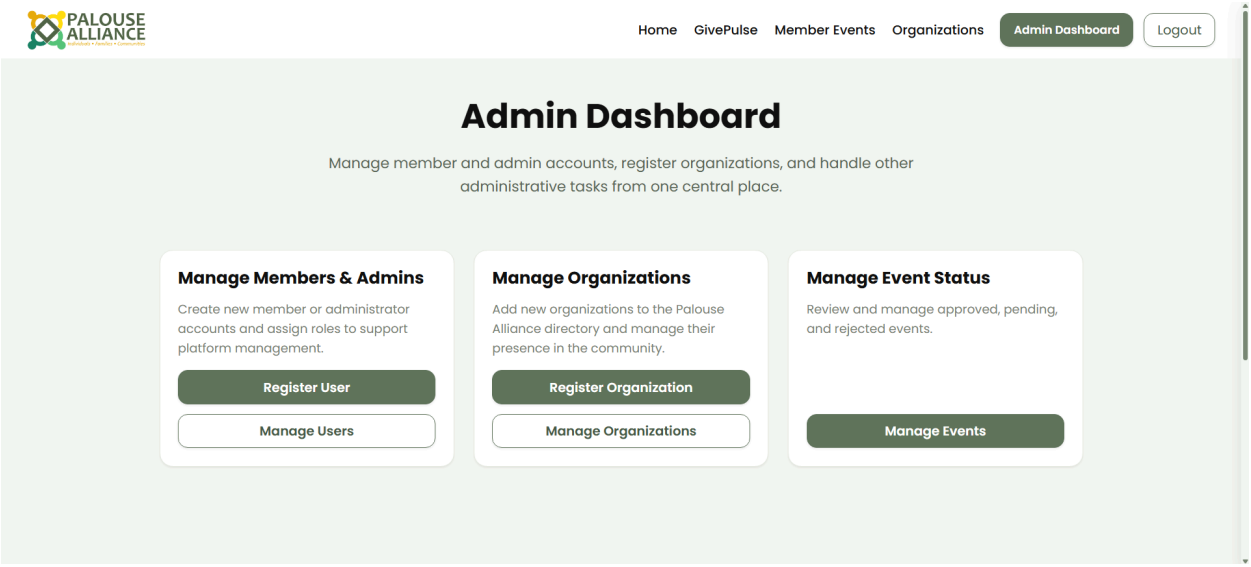


Figure 19: Admin Moderation Dashboard overview restricted to system administrators.

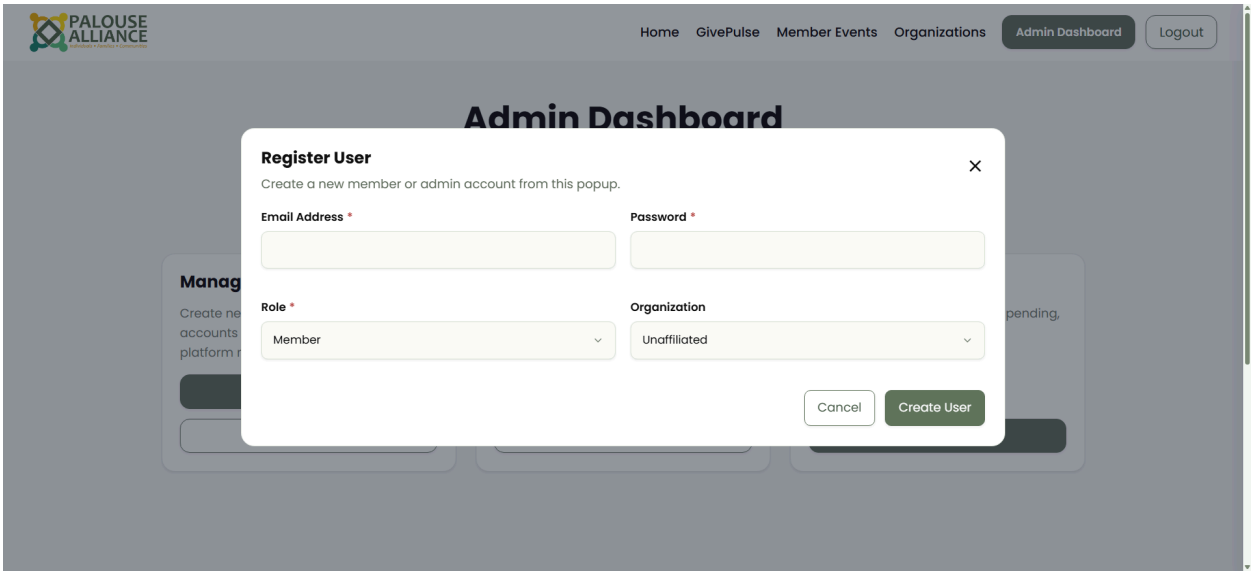


Figure 20: User Registration modal accessible within the administrative interface.

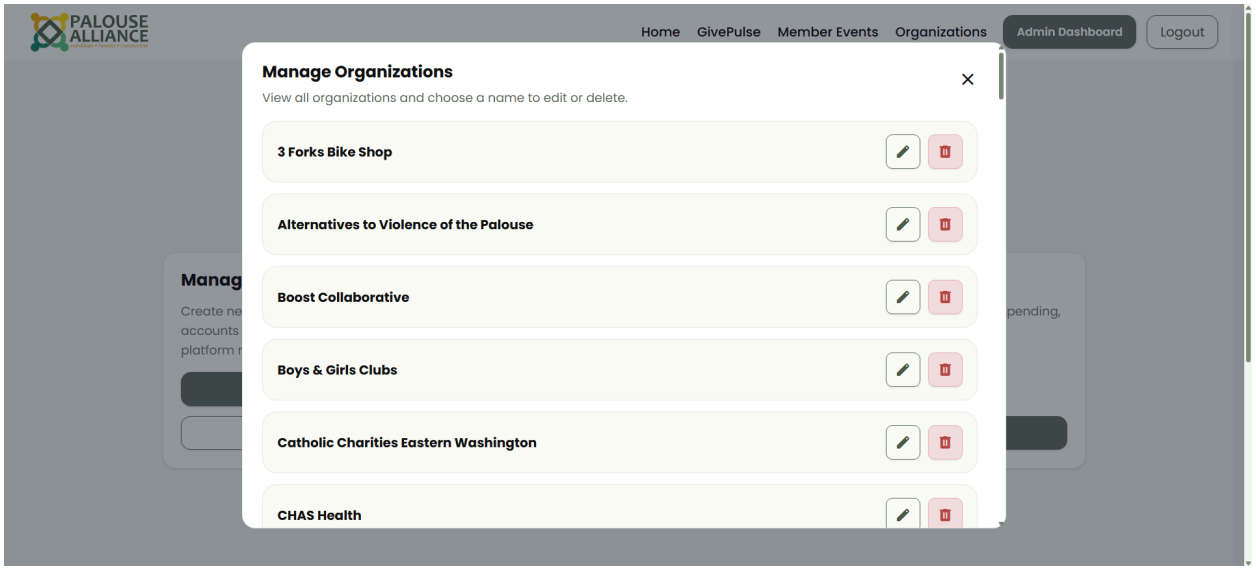


Figure 21: Member Organization editing interface for administrators.

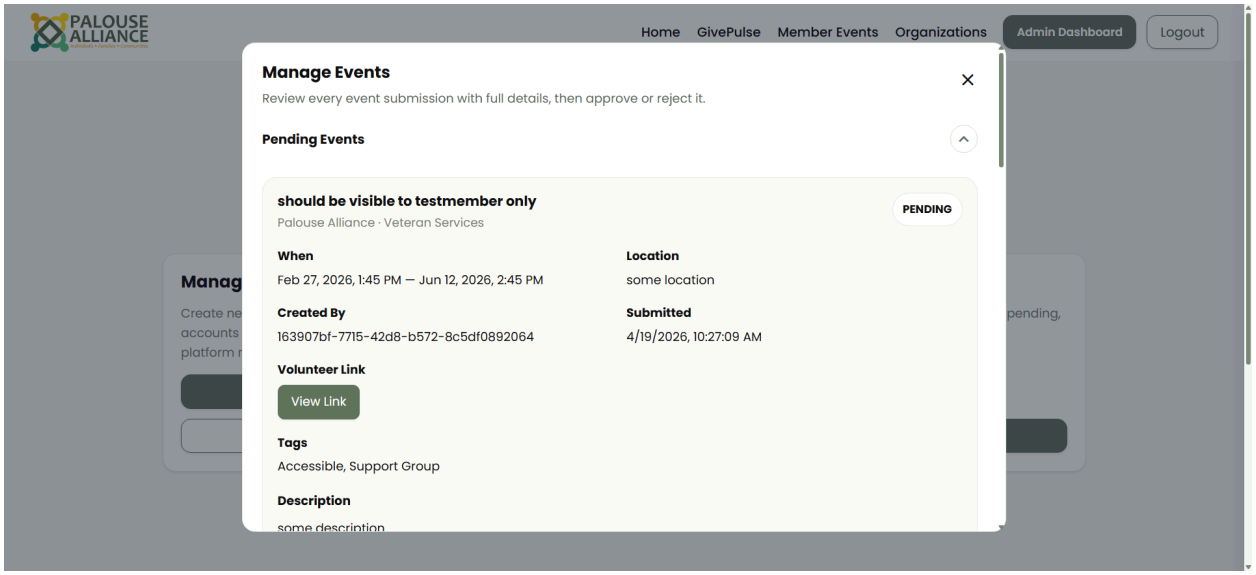


Figure 22: Moderation action controls with Approve and Reject options for submitted member events.

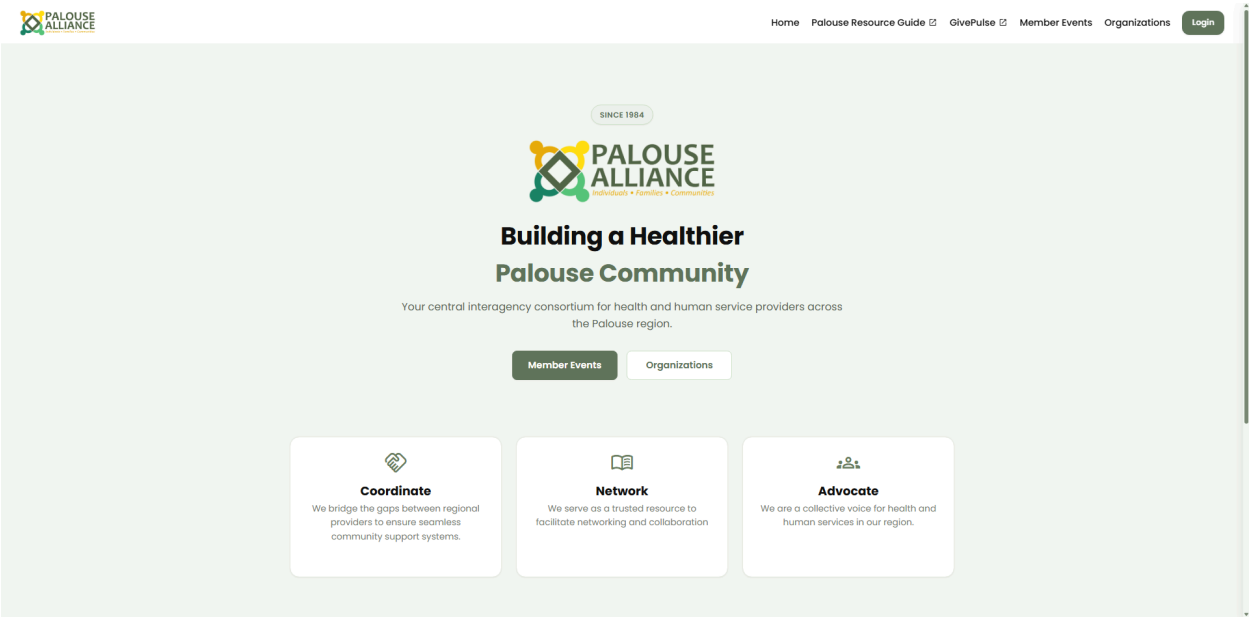


Figure 23: The landing page, shown to all users and the public.

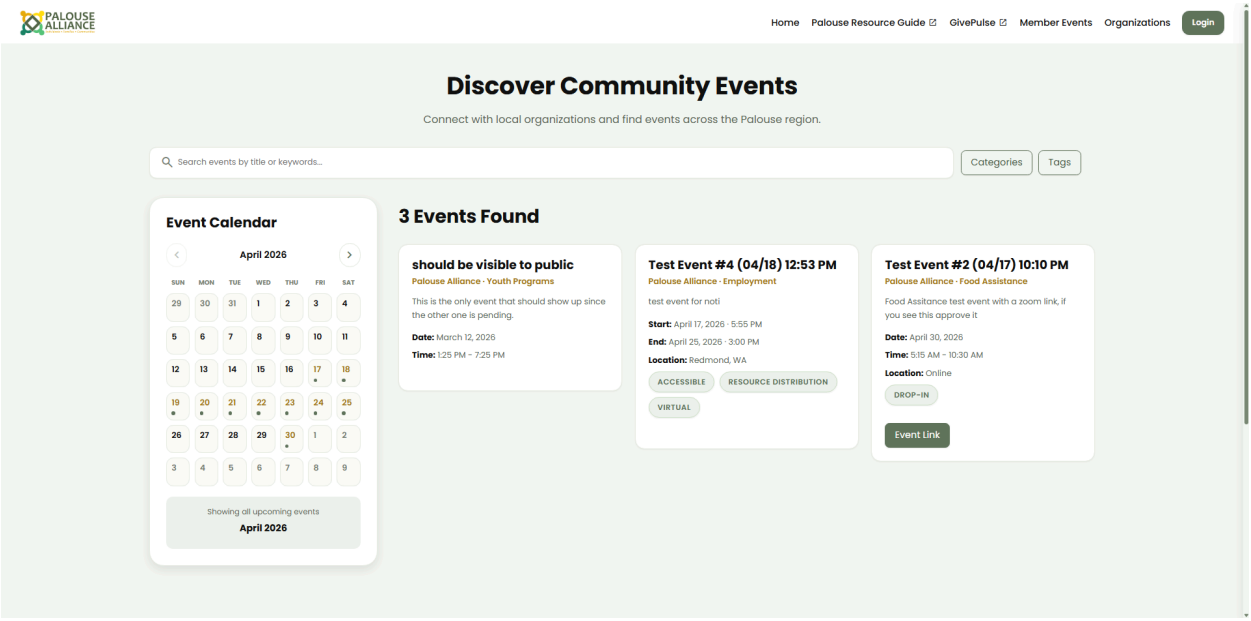


Figure 24: The events page, visible to the public.

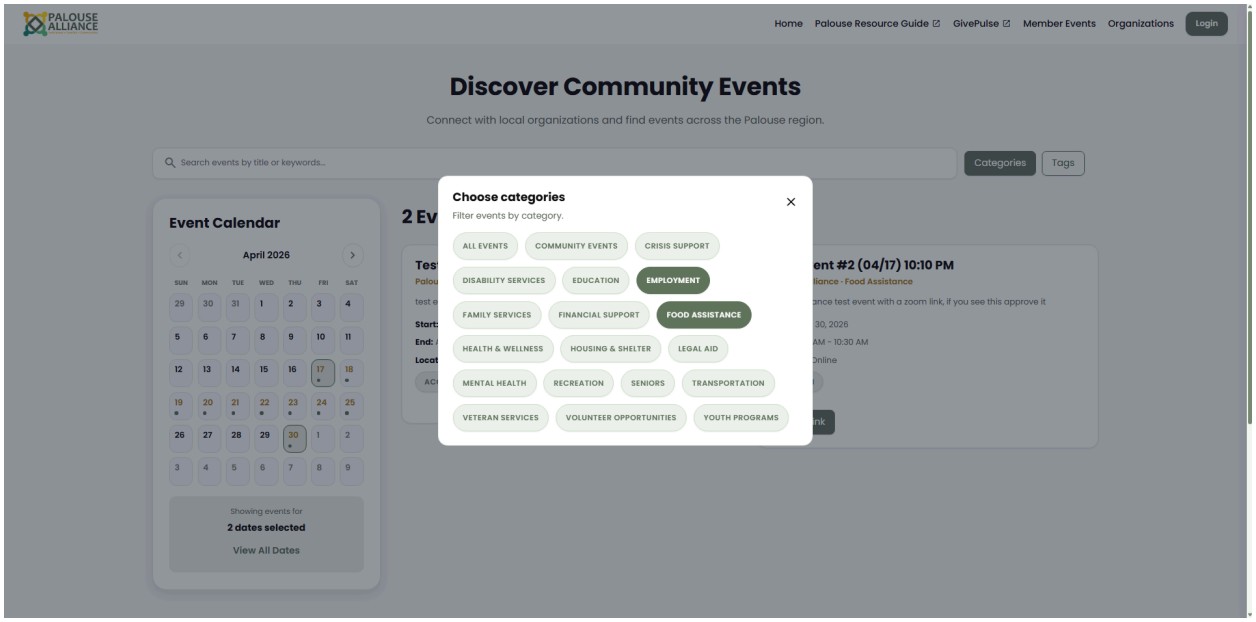


Figure 25: Categories filtering pop-up, available for all users, including tags and search bar filters.

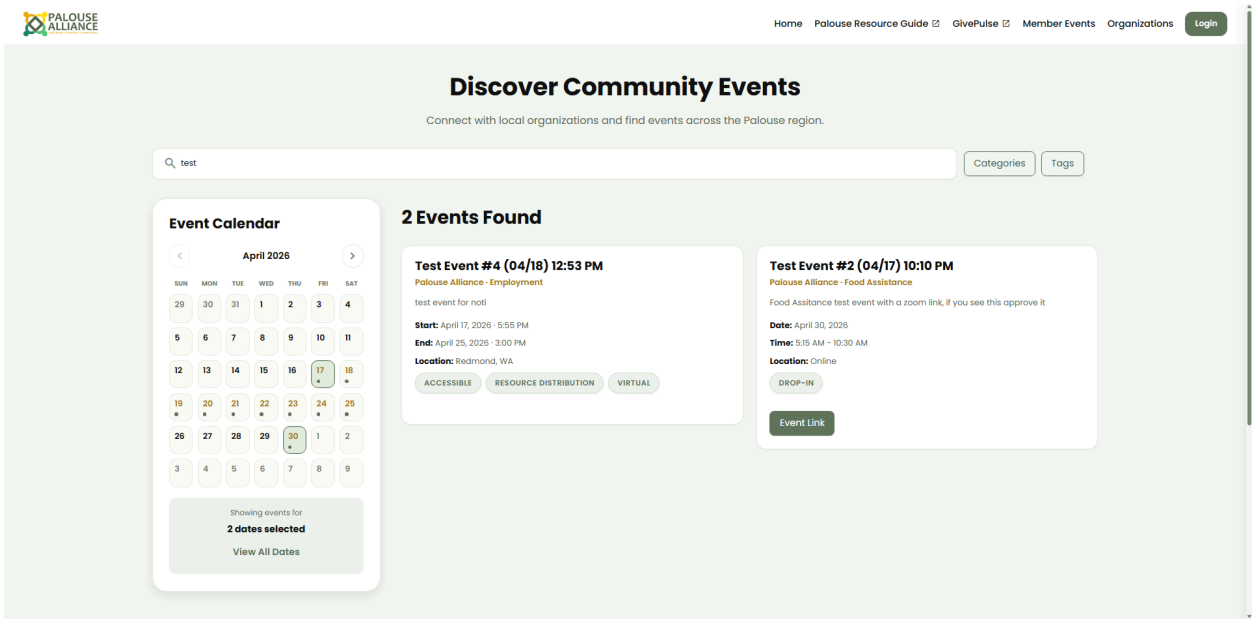


Figure 26: Calendar date filtering supporting both single-day and multi-day range selections.

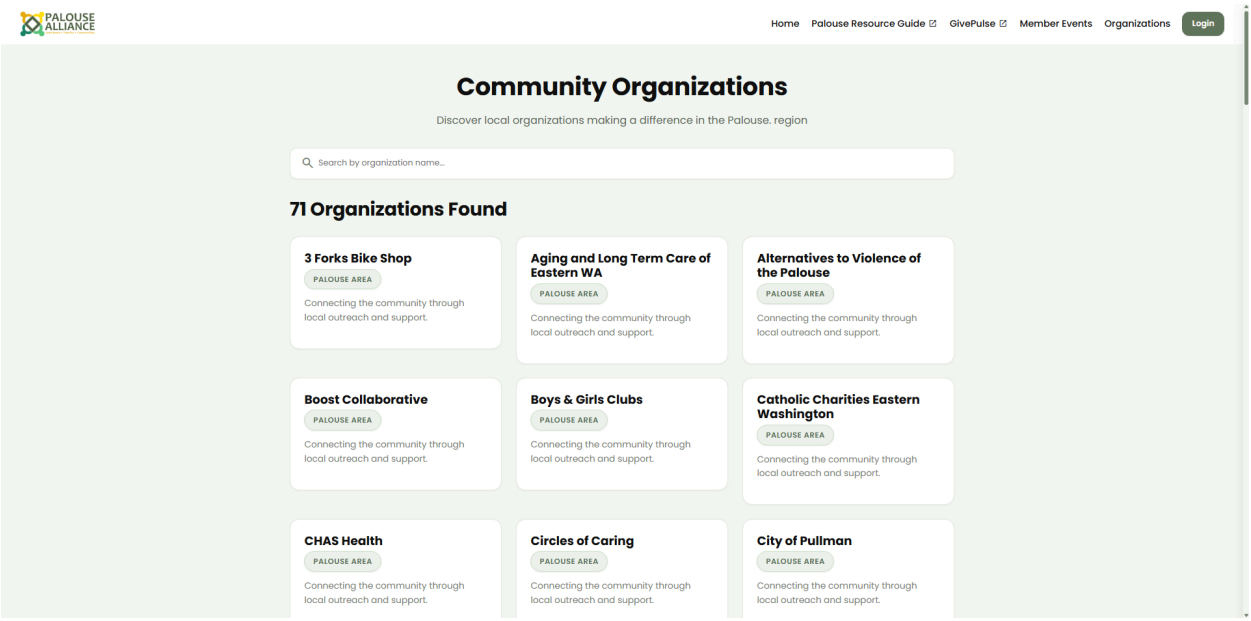


Figure 27: The organizations directory page, visible to the public.

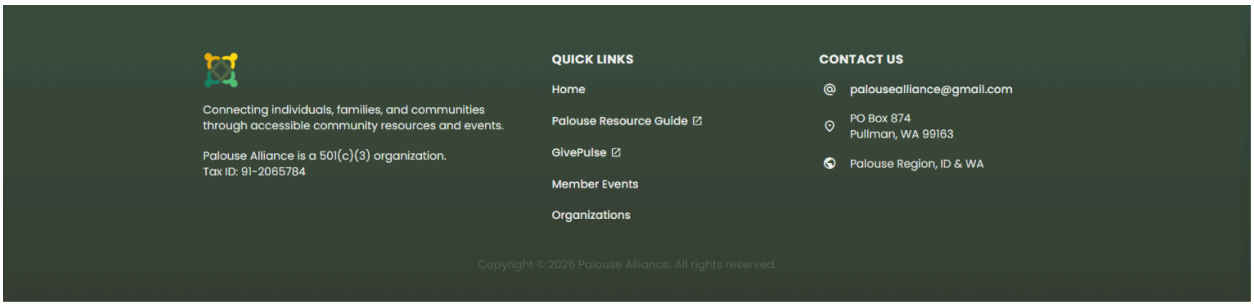


Figure 28: Platform footer containing navigation links and community disclaimers.

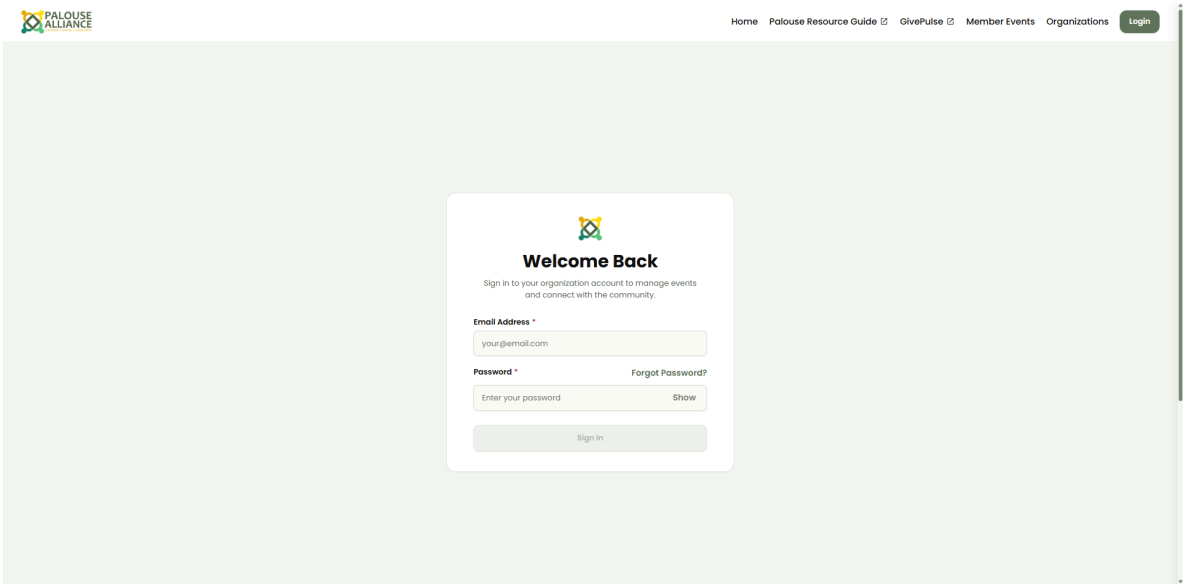


Figure 29: Login page for registered users seeking account access.

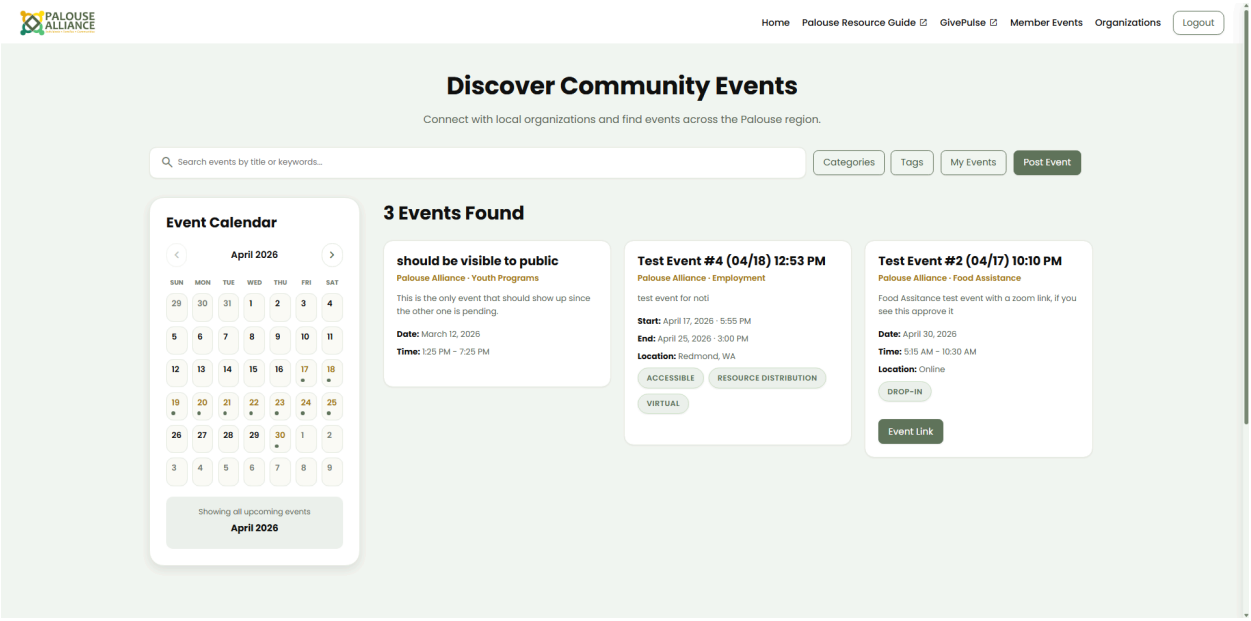


Figure 30: The events page viewed from an active, authenticated member session.

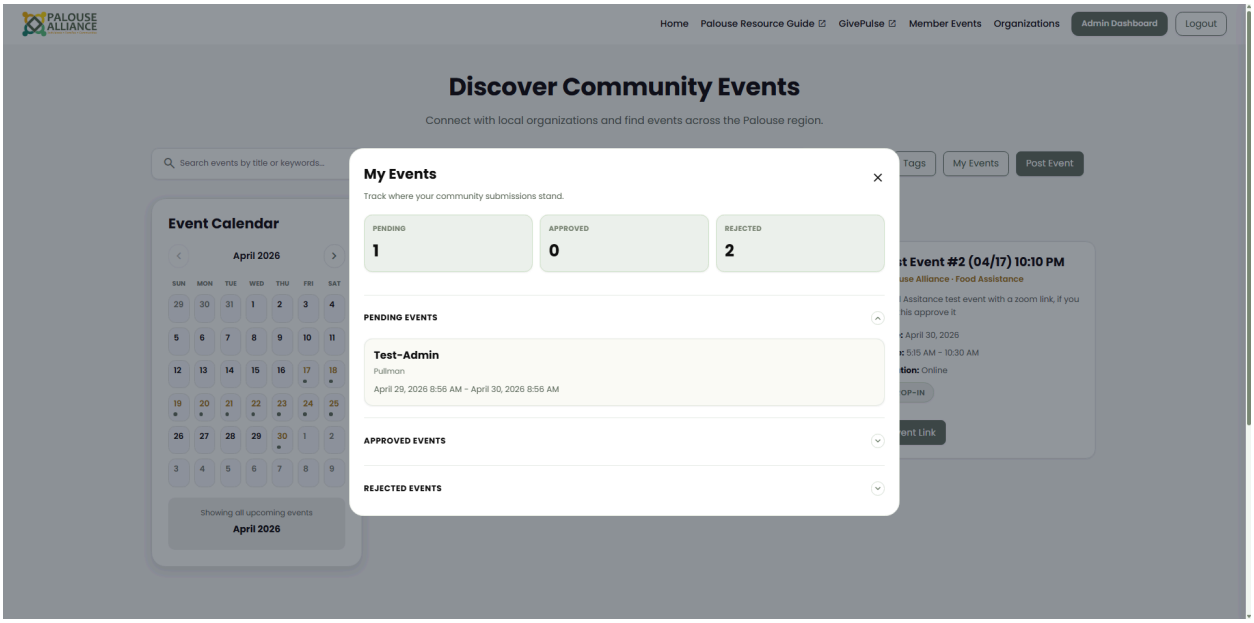


Figure 31: "My Events" pop-up showing an active pending event submission.

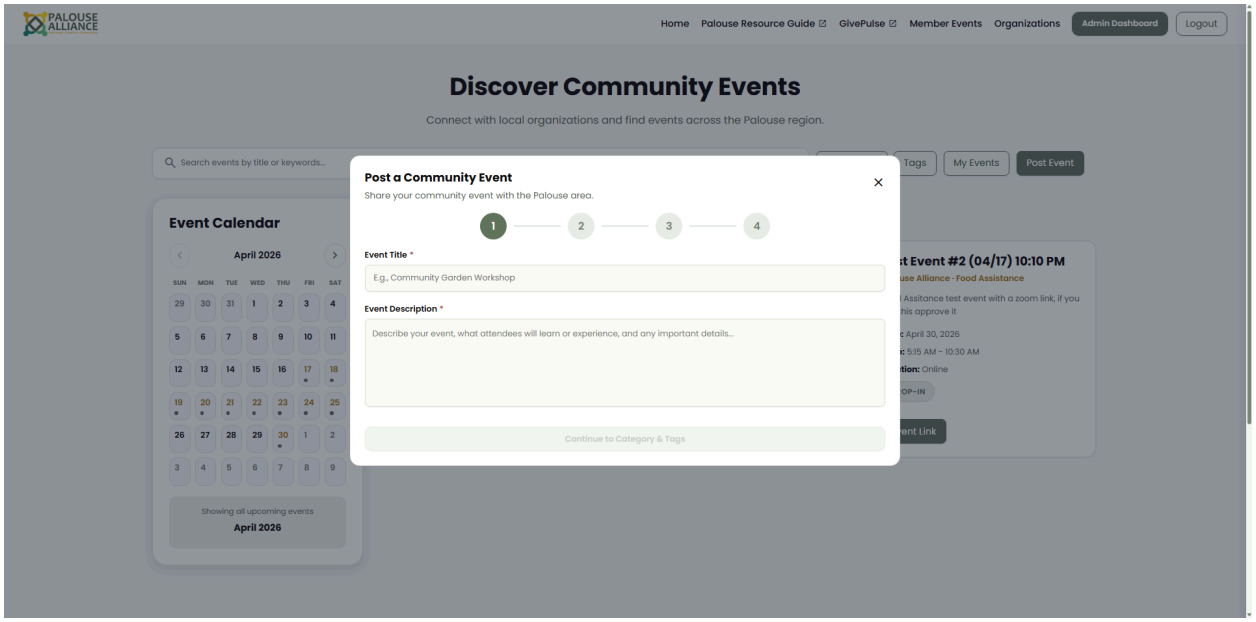


Figure 32: Step 1 of the event posting workflow, capturing title and description.

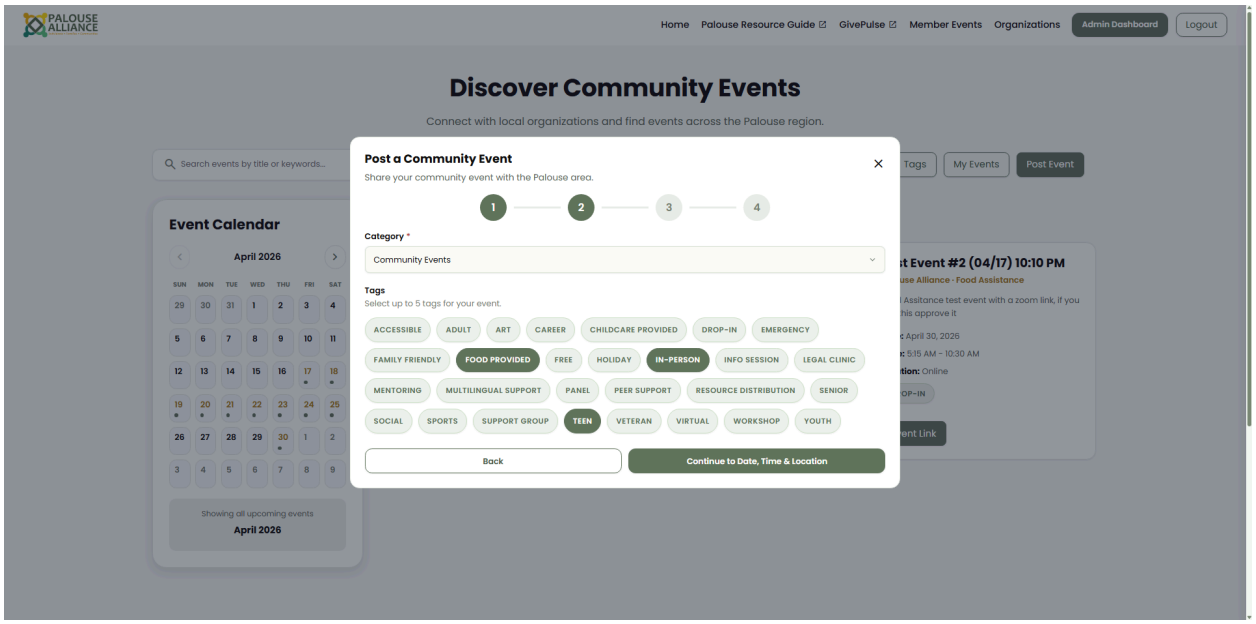


Figure 33: Step 2 of the event posting workflow, selecting categories and tags.

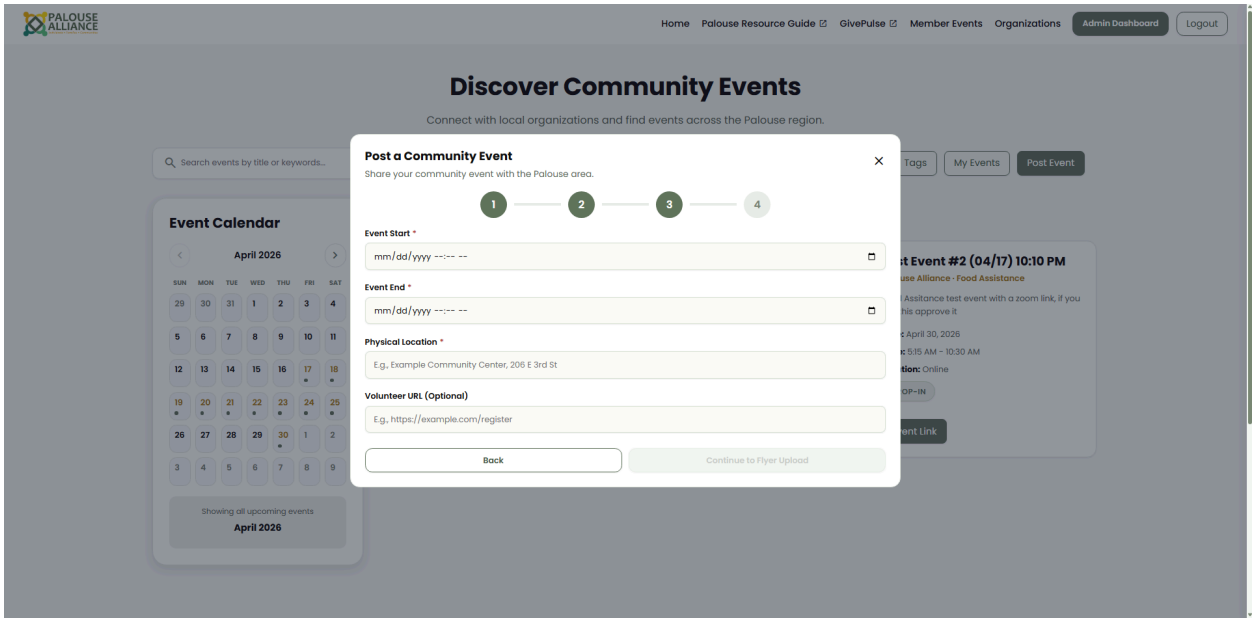


Figure 34: Step 3 of the event posting workflow, specifying date, time, and location details.

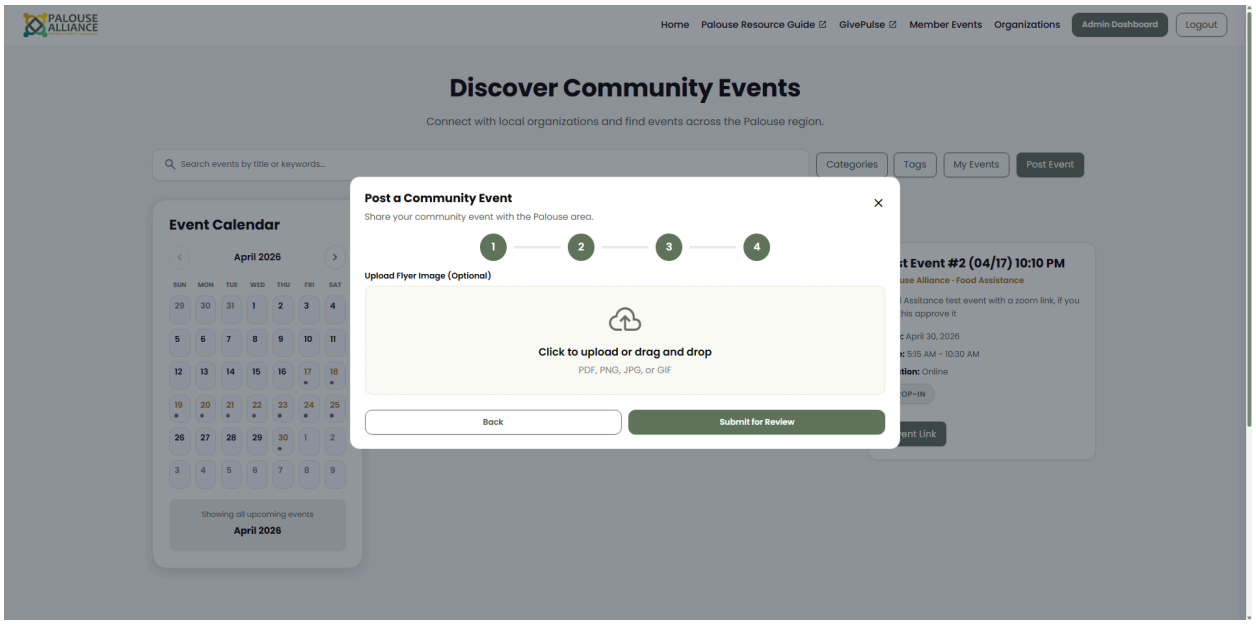


Figure 35: Step 4 of the event posting workflow, handling optional event flyer uploads.

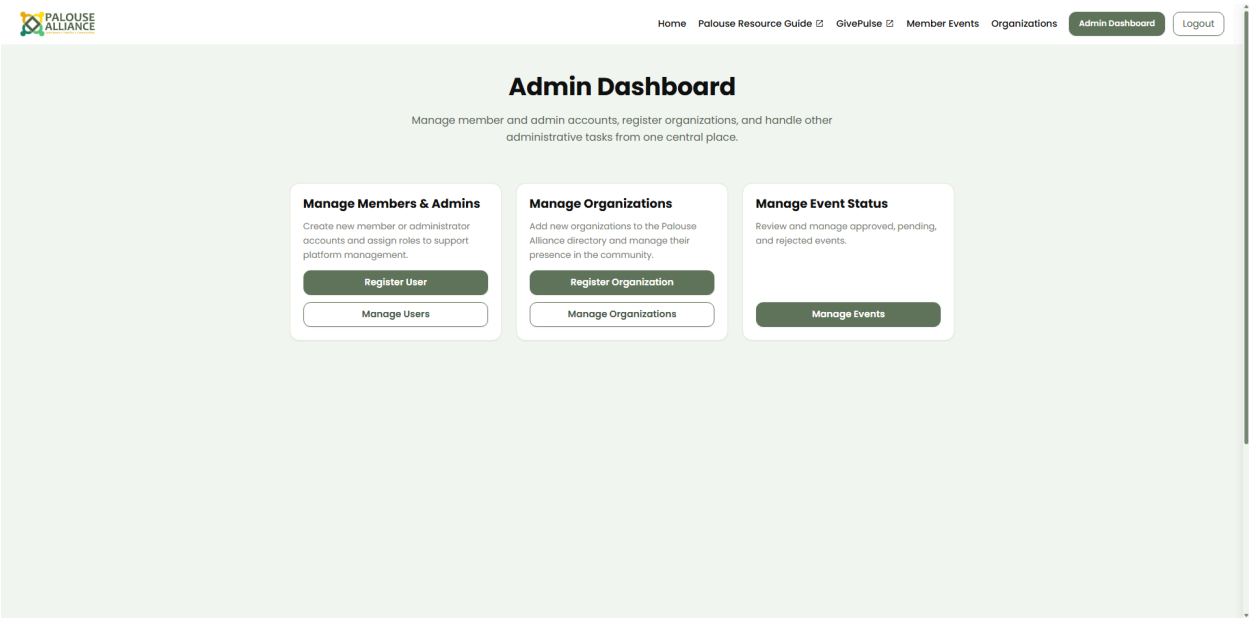


Figure 36: The administrative management dashboard.

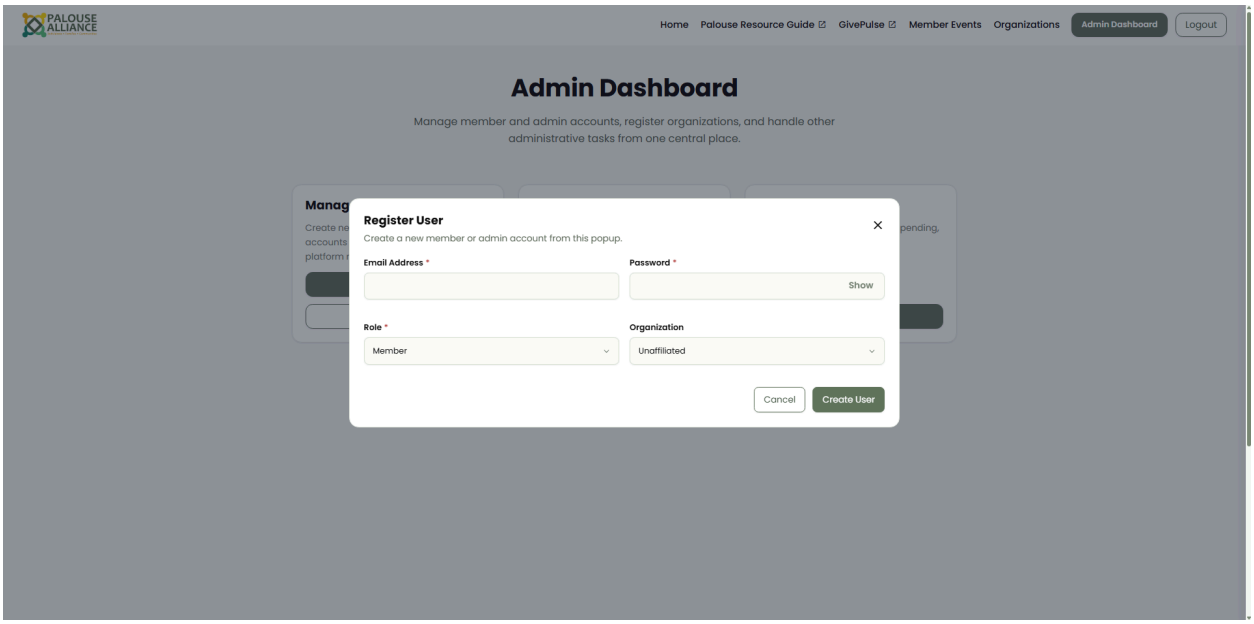


Figure 37: User account creation modal within the administrative suite.

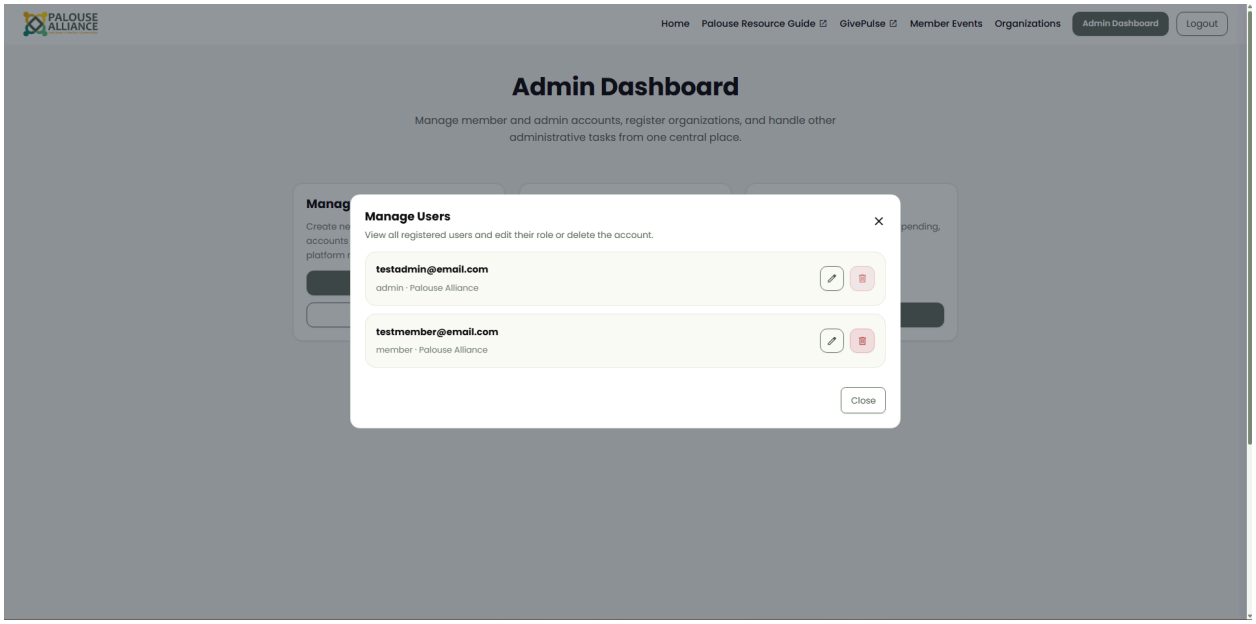


Figure 38: Member account roster allowing administrators to update or remove users.

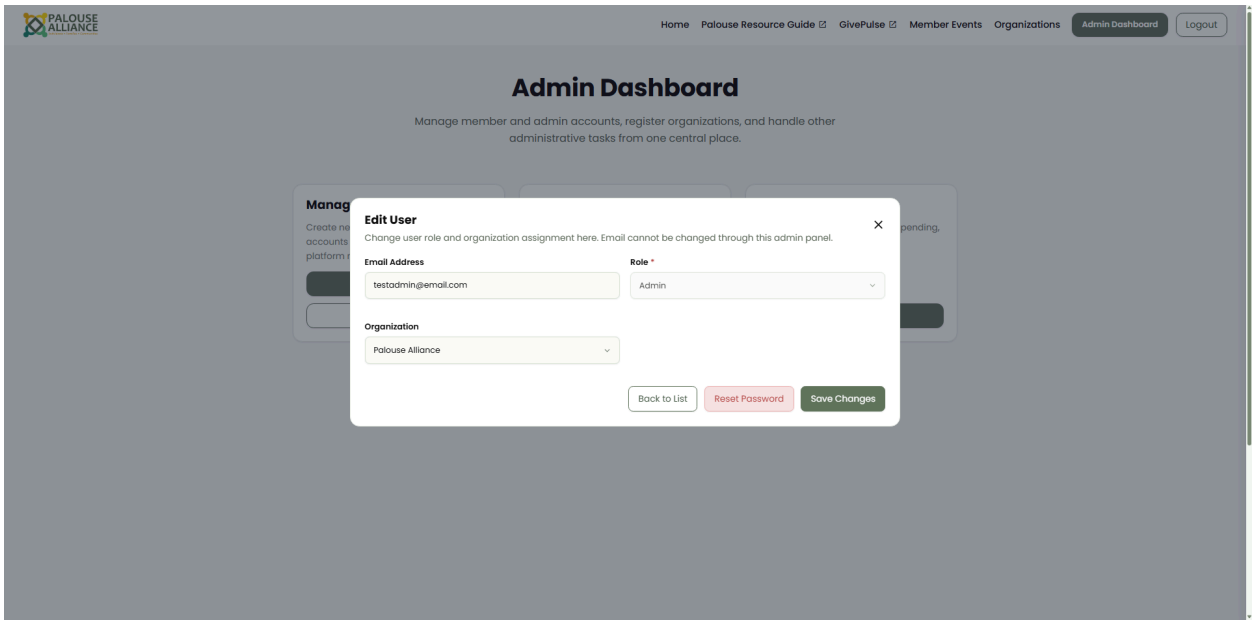


Figure 39: User profile modification form for administrators.

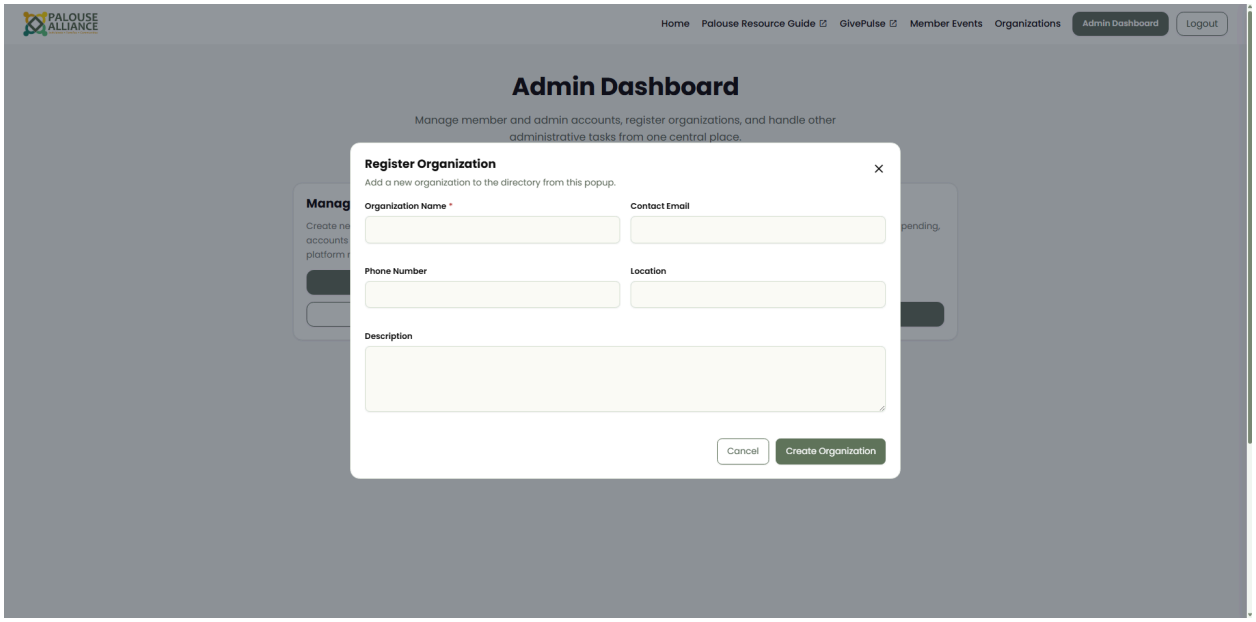


Figure 40: Community organization registration form for administrators.

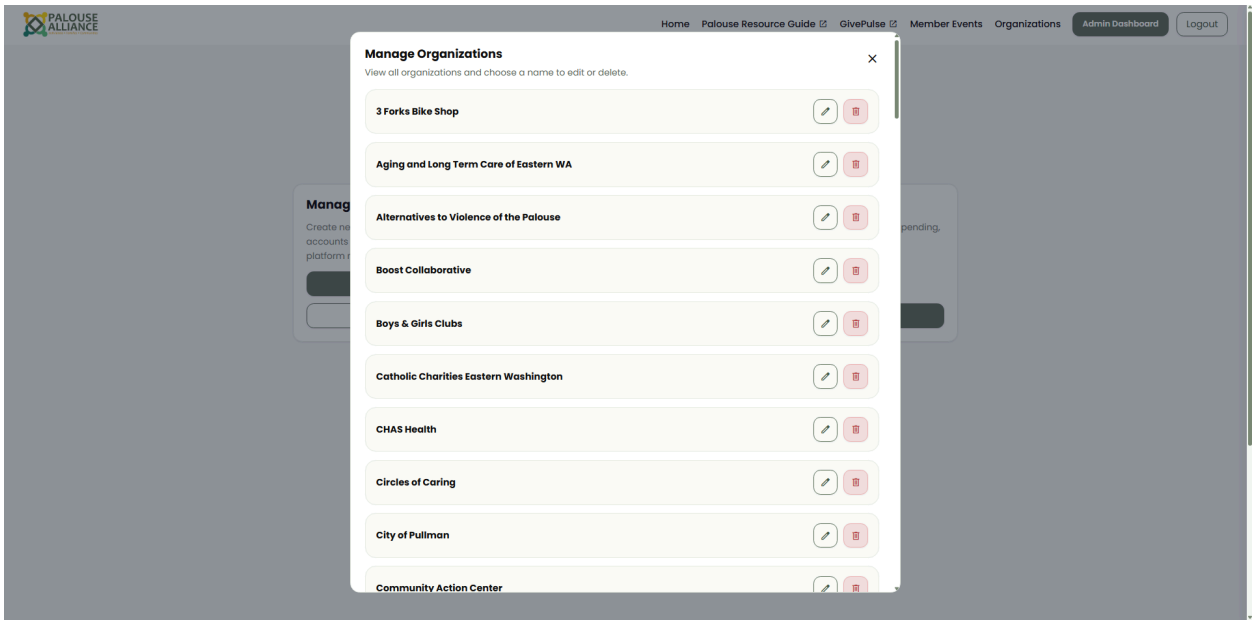


Figure 41: Master organization roster enabling administrative updates or deletions.

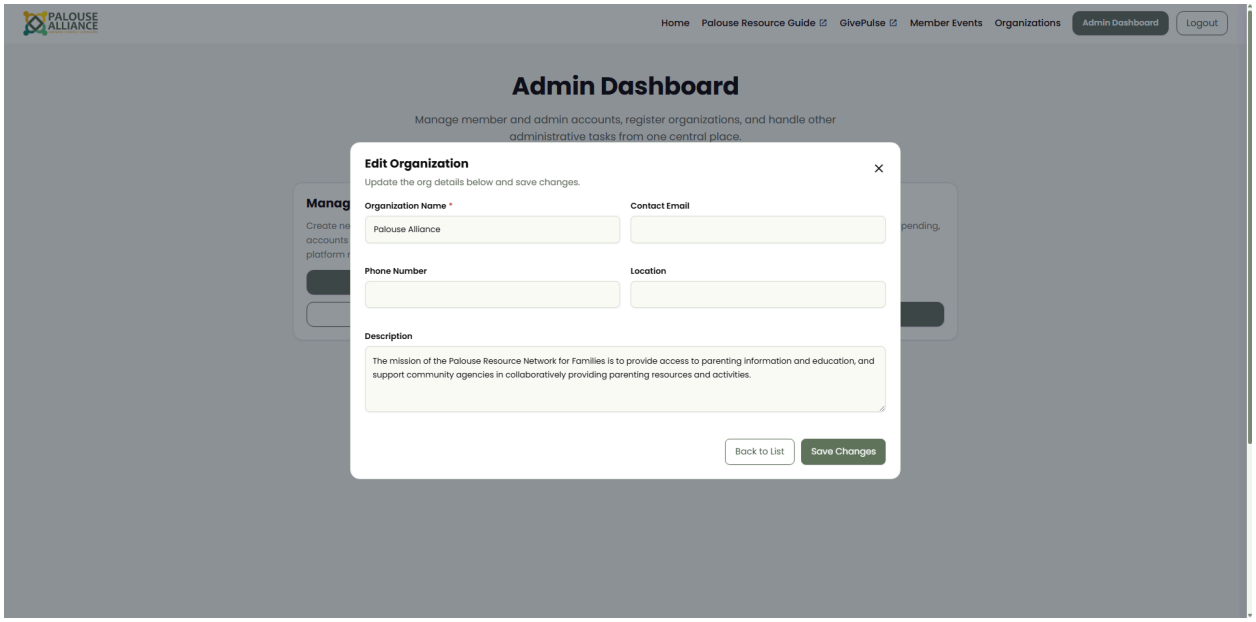


Figure 42: Organization profile editing modal for updating contact data and descriptions.

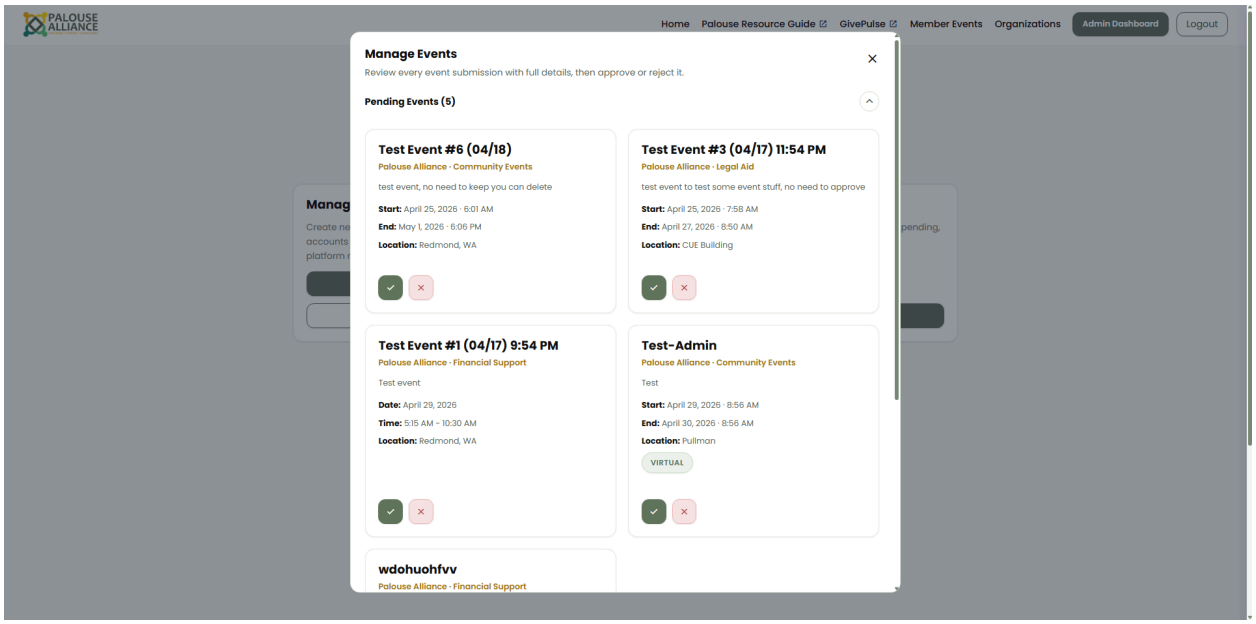


Figure 43: Pending event submissions queue for administrative review and approval.

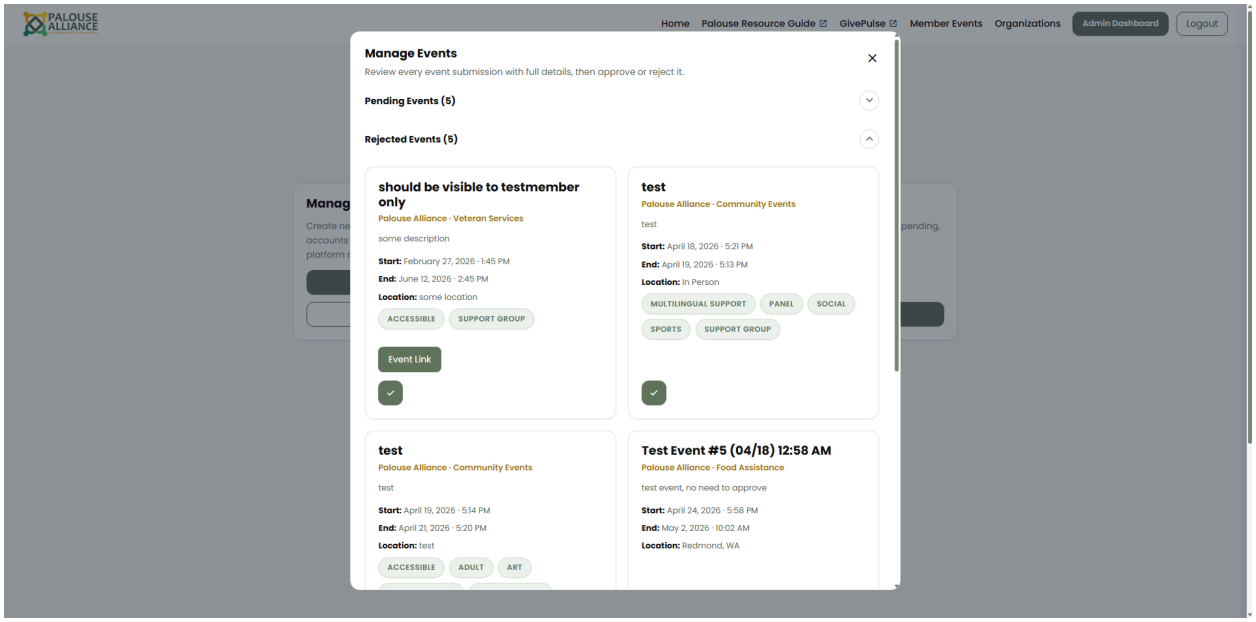


Figure 44: Rejected submissions archive allowing administrators to re-evaluate items.

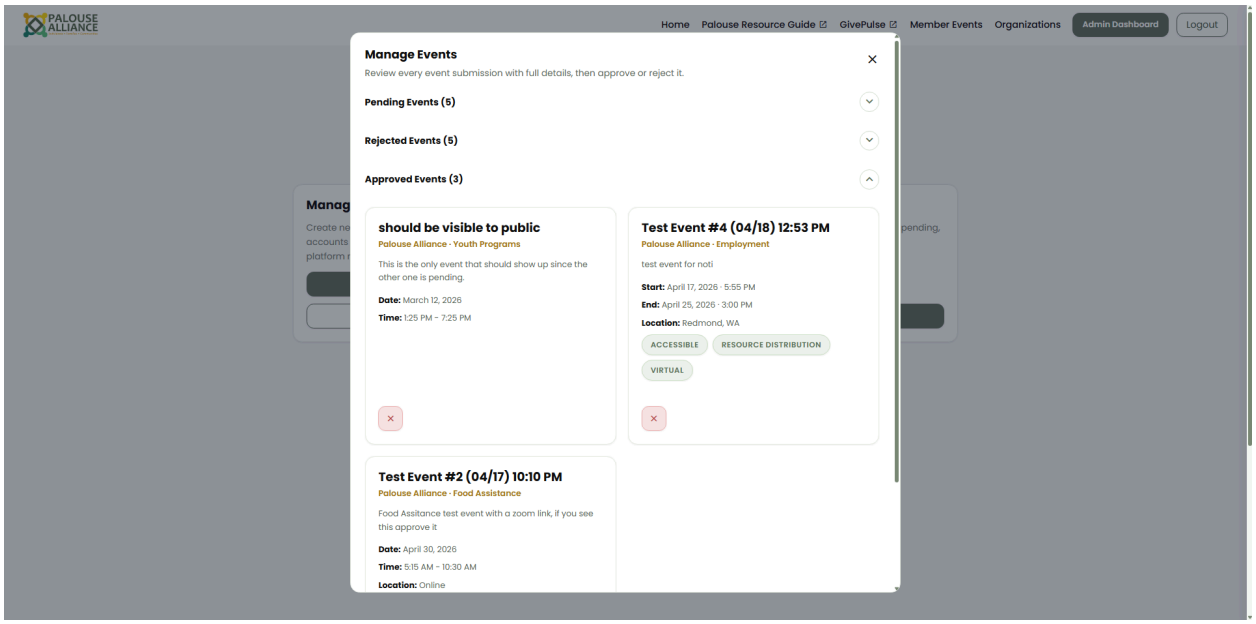


Figure 45: Published event registry allowing administrators to revoke or remove active listings.

VII.5. System Evolution

The Palouse Alliance Community Resource Platform relies on a highly modular architecture designed for continuous adaptation. Technology standards and community requirements change rapidly over time. The platform addresses these shifts through a decoupled system design. This approach ensures the application remains viable over many years. Complete system rewrites are unnecessary when user demands increase. The cloud infrastructure scales compute and storage resources automatically to handle higher web traffic and larger media uploads.

The frontend utilizes React to create an adaptable user interface. Developers can introduce new features without breaking existing functionalities. Future updates might include interactive venue maps or dedicated volunteer coordination dashboards. The component library allows developers to build these new views rapidly. The system maintains visual consistency across all new pages.

Community needs dictate the types of resources the platform must host. The database schema accommodates new data models easily. The system currently focuses on standard event listings. Future iterations will support recurring educational classes and crisis intervention hotlines. The platform will also integrate virtual meeting links and mutual aid directories.

The Palouse Alliance must exchange data seamlessly with local partner organizations. The backend infrastructure supports the addition of new application programming interfaces. These data endpoints will enable synchronization with university schedules and municipal community portals. The platform will output standard data formats to allow local media outlets to syndicate event feeds directly on their own websites.

Digital standards and regulatory requirements present critical change points. The platform utilizes automated testing pipelines to evaluate accessibility compliance continuously. The system adapts to updated web accessibility guidelines to ensure equitable access for all users. Managed backend services provide enterprise security updates automatically. The platform enforces strict data retention rules to align with emerging data privacy regulations.

Developer turnover and technological debt pose risks to platform stability. The project requires strict operational hygiene and comprehensive code documentation. Automated continuous integration pipelines test every new code contribution. These deployment workflows guarantee that future modifications deploy safely. The development team can update branding elements or upgrade software dependencies without introducing production errors.

VIII. Product Delivery Status

The Palouse Alliance Community Resource Platform has reached an advanced, deployable implementation state. The core functional requirements for public discovery, user authentication, multi-step submission, and administrative content moderation are fully implemented and integrated.

Feature Area	Delivery State	Verification Method
Public Events Calendar & Discovery	Complete	Vitest, Playwright, UAT
Keyword Search & Category Filters	Complete	Automated Component Tests
Organization Directory & Contacts	Complete	Automated Tests, UAT
Supabase Auth & Role-Based Routes	Complete	End-to-End Tests
4-Step Guided Event Submission	Complete	Integration Tests, UAT
“My Events” Member Drawer	Complete	Automated Component Tests
Admin Moderation (Approve/Reject)	Complete	Functional Scenarios
User & Organization Administration	Complete	Manual & Schema Testing
Automated 30-Day Database Cleanup	Complete	Backend Cron / Trigger
GivePulse Volunteer Integration	Complete	UI Linkage Verification
Custom Domain Deployment	In Progress	Sprint 5 Handoff
Email Notifications on Publishing	In Progress	Edge Function Pipeline
Administrative Analytics Engine	Depriortized	Per Client Request

IX. Conclusions and Future Work

IX.1. Limitations and Recommendations

While the platform fulfills core requirements, practical operational constraints remain:

- File Upload Constraints: Flyer uploads accept image files up to standard browser payloads. High-resolution print files require client-side compression before transfer to preserve **Supabase** storage quotas.
- Operational Recommendations: We recommend scheduling automated weekly database backups beyond default **Supabase** retention schedules. We also recommend establishing a periodic administrative audit to verify that partner organization contact emails and phone numbers remain active.

IX.2. Future Work

Our primary goal for upcoming development cycles is to advance the platform from an operational release into a fully polished, stakeholder-ready system deployed under the official custom domain. With core browsing, event submission, and administrative moderation established, the next phase focuses on refining workflows, expanding automation, and ensuring the platform remains easy to maintain for Alliance partners over time.

A key priority is enhancing the event lifecycle and submission management. While administrators can already review, approve, and reject submissions, we plan to give members the ability to edit rejected events directly within their dashboard and resubmit them with revision notes. We also plan to refine the organization directory so affiliated members can submit verified profile updates through structured forms, reducing manual administrative overhead while preserving accurate community records.

Technical improvements will focus on external data integration and media performance. We aim to upgrade the current GivePulse volunteer section into a direct API integration that displays live volunteer openings and registration counts within event cards. In addition, we plan to implement automated image compression for flyer uploads during the submission process, ensuring fast page load speeds on mobile networks without sacrificing visual quality.

Finally, we will expand test automation and accessibility compliance across the entire platform. Building on our existing Vitest and React Testing Library suites, we will add end-to-end integration tests that simulate full user journeys like account login, multi-step event posting, and administrative reviews. We will also conduct continuous accessibility evaluations to ensure all layouts, search tools, and form controls remain fully compliant with WCAG 2.2 standards across desktop, tablet, and mobile devices.

X. Acknowledgements

- Allison Rockwell and Christine Oakley: For serving as our primary project clients, consolidating feedback from the Palouse Alliance membership, and helping define the functional priorities of the platform.
- Palouse Alliance Member Organizations: For contributing sample program data, testing organization profile cards, and evaluating directory readability.
- School of Electrical Engineering and Computer Science at Washington State University: For providing the instructional framework, computing resources, and faculty oversight that supported this capstone project.

XI. Glossary

AWS	Amazon Web Services; A cloud computing platform that offers scalable services like storage, databases, and networking operations.
CRUD	Create, Read, Update, Delete; basic database operations.
Dependabot	Automated software dependency management tool associated with GitHub. Helps developers keep their project libraries/packages up-to-date and secure.
GivePulse	Platform designed to help individuals and organizations find and track volunteer opportunities.
GitHub Actions	A continuous integration and continuous deployment platform that allows us to automate software development workflows within our GitHub repository.
HTML	HyperText Markup Language; standard markup language used to create the structure of webpages.
HTTP(S)	HyperText Transfer Protocol; standard, unsecured protocol for sending and receiving data though a web browser.
k6	Open-source testing tool used to test performance, reliability, and API behavior.
Lifecycle	Structured, iterative process for planning, designing, building, testing, and deploying high quality software efficiently.
Modular Architecture	Design approach that structures a program into independent and interchangeable parts called modules.
SEO	Search Engine Optimization; the process of improving a website to increase its visibility in organic search results, driving unpaid, relevant traffic.
SQL	Structured Query Language; standardized programming language used to manage and manipulate data in databases.
Supabase	An open-source Backend-as-a-Service platform that provides the project's PostgreSQL database, user authentication, and cloud storage functionality.
UAT	User acceptance testing. Final phase of testing to determine acceptance criteria.
Use Cases	Description of interactions between an actor and system.
WCAG	Web Content Accessibility Guidelines; standards for making web content accessible.

XII. References

[1] U.S. General Services Administration, “Section508.gov,” [Online]. Available: https://www.section508.gov/. [Accessed: Feb. 15, 2026].

[2] American Public Human Services Association (APHSA), “Why Digital Accessibility Benefits Everyone,” Aug. 06, 2025. [Online]. Available: https://aphsa.org/resources/why-digital-accessibility-benefits-everyone-access-beyond-disabilities-thecatalyst/. [Accessed: Feb. 15, 2026].

[3] W3C Web Accessibility Initiative (WAI), “Web Content Accessibility Guidelines (WCAG) Overview,” [Online]. Available: https://www.w3.org/WAI/standards-guidelines/wcag/. [Accessed: Feb. 15, 2026].

[4] World Wide Web Consortium (W3C), “Web Standards,” [Online]. Available: https://www.w3.org/standards/. [Accessed: Feb. 15, 2026].

[5] W3C, “WAI-ARIA Overview,” [Online]. Available: https://www.w3.org/WAI/standards-guidelines/aria/. [Accessed: Feb. 15, 2026].

[6] U.S. Department of Justice, “ADA Requirements: Effective Communication,” [Online]. Available: https://www.ada.gov/resources/effective-communication/. [Accessed: Feb. 15, 2026].

[7] IEEE, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Std 830-1998, 1998.

[8] ISO/IEC/IEEE, “Systems and software engineering, Software life cycle processes,” ISO/IEC/IEEE 12207:2017, 2017.

[9] IEEE, “IEEE Standard for Software Design Descriptions,” IEEE Std 1016-2009, 2009.

[10] IEEE, “IEEE Standard for Software Test Documentation,” IEEE Std 829-2008, 2008.

[11] W3C, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, 2023.

[12] W3C, “WAI-ARIA 1.2, Accessible Rich Internet Applications,” W3C Recommendation, 2021.

[13] OWASP, “Application Security Verification Standard (ASVS) v4.0.3,” OWASP Foundation, 2021.

[14] T. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” IETF RFC 8446, Aug. 2018.

[15] European Parliament and Council, “Regulation (EU) 2016/679, General Data Protection Regulation (GDPR),” 2016.

[16] U.S. Congress, “Controlling the Assault of Non-Solicited Pornography And Marketing Act of 2003 (CAN-SPAM Act),” 2003.

[17] Lucid Software Inc., “ER Diagram Symbols and Notation,” Lucidchart Blog, [Online]. Available: https://www.lucidchart.com/blog/er-diagram-symbols-and-notation.

XIII. Appendix

A. Additional Use Cases

A.1. [UC-6] Manage Deployment and Branding

Use Case	Manage Deployment and Branding
Actors	Admin, Technical Maintainer
Pre Condition	Admin/tech maintainer is authenticated and has access to platform settings.
Post Condition	Custom domain, branding, and **SEO** options updated.
Main Flow	- Admin navigates to deployment/configuration settings. - Admin updates domain name, branding theme, or **SEO** metadata. - The platform reflects the updates appropriately.
Alternative Flow	- Invalid values, system prompts user for correction.
Related Requirements	FR-12: Custom Domain Name Support FR-13: Branding and Theming FR-15: Search Engine Display

B. User Stories

This section presents user stories that show goals and interactions of the Palouse Alliance Community Resource Platform from different user perspectives. Each user story is in Gherkin syntax to better specify user intent, relevant context, actions taken, and desired outcomes.

B.1. [US-1] User Login

As a member,
I want to log in to my account
so that I can access member-only features and manage my listings.

Feature: User Login
Scenario: Valid member logs in
    Given the member is on the login page
    When they enter their correct username and password
    Then they should be redirected to their account dashboard.

B.2. [US-2] Search for Services or Events

As a member,
I want to search for community services and events
so that I can find relevant support or activities quickly.

Feature: Service or Event Search
Scenario: User searches for a service or event
Given the user is on the homepage
When they enter a search term in the search bar
Then relevant services or event listings should be displayed.

B.3. [US-3] Category Filtering

As a visitor,
I want to filter services and events by category or tag
so that I can easily narrow down the results to what I need.

Feature: Category Filtering
Scenario: Filtering by category or tag
Given the user is viewing services or events
When they select a specific category or tag filter
Then only relevant services or events should be shown.

B.4. [US-4] Post Event or Flyer

As a member,
I want to submit a new event or flyer
so that our organization can share information with the community.

Feature: Event/Flyer Posting
Scenario: Member posts an event or flyer
Given the member is logged in and on the submission page
When they provide event or flyer details and submit
Then the submission should be sent for admin approval.

B.5. [US-5] Admin Approves Submissions

As an admin,
I want to review and approve submitted events and flyers before publication
so that only appropriate content appears publicly.

Feature: Submission Approval
Scenario: Admin approves a new submission
Given the admin is reviewing pending submissions
When they approve a submission
Then the event or flyer should be published for public view.

B.6. [US-6] View Flyers and Meetings

As a public user,
I want to view upcoming flyers, meetings, and events
so that I can stay informed about local resources and happenings.

Feature: Viewing Events
Scenario: User browses upcoming events
Given the user is on the calendar or events page
When they view the list of upcoming events
Then event details should be displayed.

B.7. [US-7] Request Organization Listing Updates

As an organization member,
I want to see clear steps on how to request edits to my organization's details
so that our profile information remains accurate and verified.

Feature: Organization Profile Update Guidance
Scenario: Member views instructions to request updates
Given the member is logged in and viewing their affiliated organization's card
When they access the organization details section
Then the card displays explicit guidance directing them to submit updated information to an administrator for verification.

B.8. [US-8] Update Platform Configuration

As an admin or technical maintainer,
I want to update branding and **SEO** settings
so that the platform aligns with our organization's needs and is discoverable online.

Feature: Platform Management
Scenario: Update settings
Given the admin is in settings
When they update the branding or **SEO**
Then the platform records and applies the changes.

B.9. [US-9] Discover Volunteer Opportunities

As a resident,
I want to browse volunteer opportunities
so that I can find ways to help my community.

Feature: **GivePulse** Integration
Scenario: Viewing available volunteer opportunities
Given the user is on the volunteer opportunities page
When they view the list of available opportunities
Then opportunities integrated from **GivePulse** should be displayed.

B.10. [US-10] View Platform Analytics

As an admin,

I want to access analytics and attendance tracking
so that I can monitor site usage and event participation.

Feature: Platform Analytics
Scenario: Admin reviews analytics dashboard
Given the admin is logged into the dashboard
When they navigate to the analytics section
Then usage statistics and attendance reports should be displayed.

C. Traceability Matrix

Functional Requirement	Use Cases	User Stories	Priority
FR-1: Search for Services and Events	UC-1: Search and Attend Events	US-2: Search for Services or Events	Level 0
FR-2: Category-Based Filtering	UC-1: Search and Attend Events	US-3: Category Filtering	Level 0
FR-3: Event Calendar Integration	UC-1: Search and Attend Events	US-2: Search for Services or Events US-6: View Flyers and Meetings	Level 0
FR-4: Member and Admin Account Management	UC-2: Upload and Approve Event or Flyer UC-3: Manage Organization Listing	US-1: User Login	Level 0
FR-5: Event and Flyer Upload and Approval	UC-2: Upload and Approve Event or Flyer	US-4: Post Event or Flyer US-5: Admin Approves Submissions	Level 0
FR-6: View Flyers and Meetings	UC-1: Search and Attend Events	US-6: View Flyers and Meetings	Level 0
FR-7: Organization and Agency Listings	UC-3: Manage Organization Listing	US-7: Request Organization Listing Updates	Level 1
FR-8: Display Contact Information	UC-3: Manage Organization Listing	US-7: Request Organization Listing Updates	Level 0
FR-9: Selective Display of Personal Information	UC-3: Manage Organization Listing	US-7: Request Organization Listing Updates	Level 0
FR-10: **GivePulse** Integration	UC-4: Volunteer Opportunity Discovery	US-9: Discover Volunteer Opportunities	Level 0
FR-11: Analytics and Attendance Tracking	UC-5: Access Community Analytics	US-10: View Platform Analytics	Level 2

Functional Requirement	Use Cases	User Stories	Priority
(Deprioritized)			
FR-12: Custom Domain Name Support	UC-6: Manage Deployment and Branding	US-8: Update Platform Configuration	Level 1
FR-13: Branding and Theming	UC-6: Manage Deployment and Branding	US-8: Update Platform Configuration	Level 1
FR-14: Responsive Web Design	UC-1: Search and Attend Events (Applies Globally)	US-2, US-6 (Applies Globally)	Level 0
FR-15: Search Engine Display	UC-6: Manage Deployment and Branding	US-8: Update Platform Configuration	Level 2
FR-16: Email Notifications for New Resources	UC-2: Upload and Approve Event or Flyer	US-5: Admin Approves Submissions	Level 0
FR-17: Event Lifecycle and Data Retention	UC-2: Upload and Approve Event or Flyer	US-5: Admin Approves Submissions	Level 0
FR-18: FAQ Page / Knowledge Base	UC-1: Search and Attend Events	US-2: Search for Services or Events	Level 1